



第6章 分治与排序(2)

——分治思想看排序



本讲内容

6.1 分治法

6.2 排序介绍

6.3 归并排序 (merge sort)

6.4 快速排序 (quick sort)

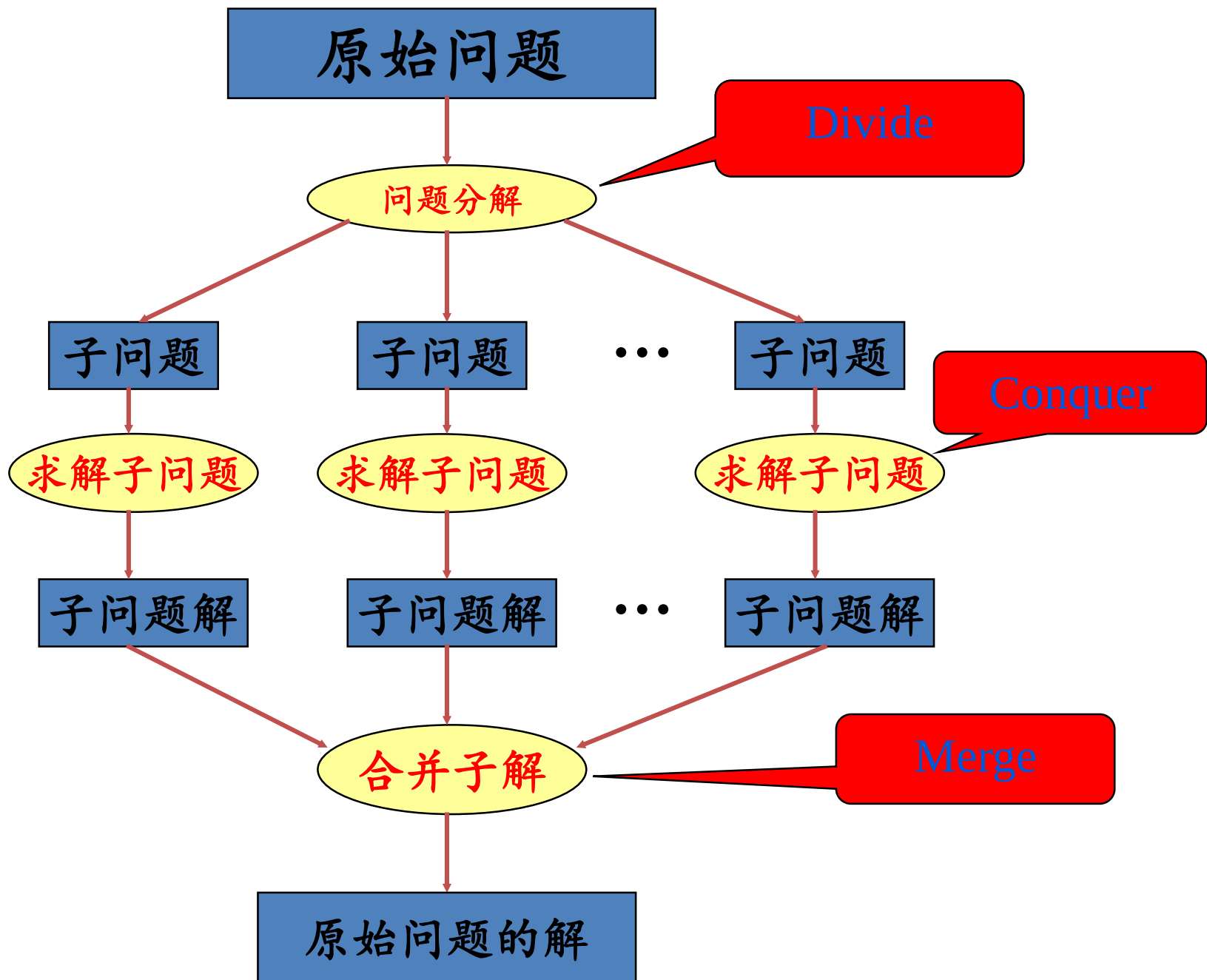
6.5 排序问题的下界

6.1 分治法

- 分治算法的设计
- 分治算法的分析

分治算法的设计

- 设计过程分为三个阶段
 - 划分（Divide）： 整个问题划分为多个互相独立的子问题
 - 求解（Conquer）： 求解各子问题（递归调用正在设计的算法）
 - 合并（Combine）： 合并子问题的解，形成原始问题的解



分治算法的分析

- 分析过程
 - 建立递归方程
 - 求解
- 递归方程的建立方法
 - 设输入大小为 n , $T(n)$ 为时间复杂性
 - 当 $n < c$, $T(n) = \theta(1)$

– Divide阶段的时间复杂性

- 划分问题为 a 个子问题。
- 每个子问题大小为 n/b 。
- 划分时间可直接得到 = $D(n)$

– Conquer阶段的时间复杂性

- 递归调用
- Conquer时间 = $aT(n/b)$

– Combine阶段的时间复杂性

- 时间可以直接得到 = $C(n)$

– 总之

$$\bullet T(n) = \begin{cases} \Theta(1) & \text{if } n < c \\ aT\left(\frac{n}{b}\right) + D(n) + C(n) & \text{otherwise} \end{cases}$$

– 求解递归方程 $T(n)$

- 使用第二章的方法

6.2 排序问题

排序是计算机内经常进行的一种操作，其目的是将一组“无序”的记录序列调整为“有序”的记录序列

例如，将下列关键字序列

52, 49, 80, 36, 14, 58, 61, 23, 97, 75

调整为

14, 23, 36, 49, 52, 58, 61, 75, 80, 97

•一般情况下，假设含 n 个记录的序列为

$$\{R_1, R_2, \dots, R_n\}$$

其相应的关键字序列为

$$\{K_1, K_2, \dots, K_n\}$$

这些关键字相互之间可以进行比较，即在它们之间存在着这样一个关系

$$K_{p1} \leq K_{p2} \leq \dots \leq K_{pn}$$

按此固有关系将式(1)的记录序列重新排列为

$$\{R_{p1}, R_{p2}, \dots, R_{pn}\}$$

的操作称作**排序**。

(张三, 89), (李四, 55), (王五, 55), (麻六, 92), (赵七, 10)

元组第一项表示姓名, 元组第二项表示成绩

根据成绩从小到大可以将这些元组排序为

(赵七, 10), (李四, 55), (王五, 79), (张三, 89), (麻六, 92)

```
Type define struct Record{  
    char name[20];  
    integer score;  
} Record;  
Record R[5];
```

$R[0] = (\text{张三}, 89), R[1] = (\text{李四}, 55), \dots, R[4] = (\text{赵七}, 10)$

内部排序与外部排序

- 若整个排序过程不需要访问外存便能完成，则称此类排序问题为内部排序
- 若参加排序的记录数量很大，整个序列的排序过程不可能在内存中完成，则称此类排序问题为外部排序
- 我们本章主要研究内部排序的各种方法

内部排序方法的分类

在排序的过程中，参与排序的记录序列中存在两个区域：有序区和无序区。内部排序的过程是一个逐步扩大记录的有序序列长度的过程

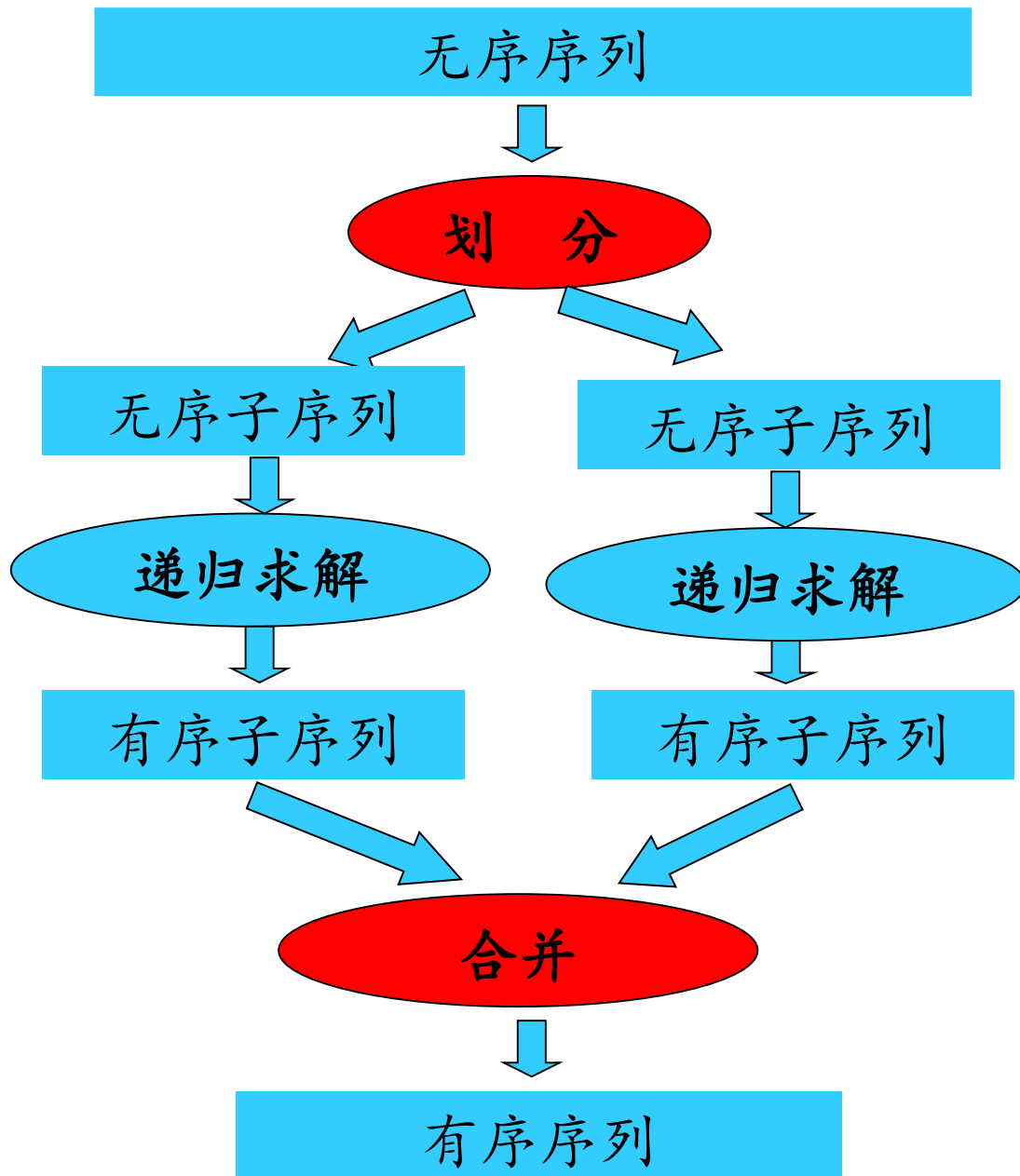
有序序列区	无序序列区
-------	-------

使有序区中记录的数目增加一个或几个的操作称为**一趟排序**

逐步扩大记录的有序序列长度的方法有下列几类：

- **插入类** 将无序子序列中的一个或几个记录“插入”到有序序列中，从而增加记录的有序子序列的长度
- **选择类** 从记录的无序子序列中“选择”关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度
- **交换类** 通过“交换”无序序列中的记录从而得到其中关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度
- **归并类** 通过“归并”两个或两个以上的记录有序子序列，逐步增加记录有序序列的长度
- 其它方法

6.3 基于分治思想的排序算法



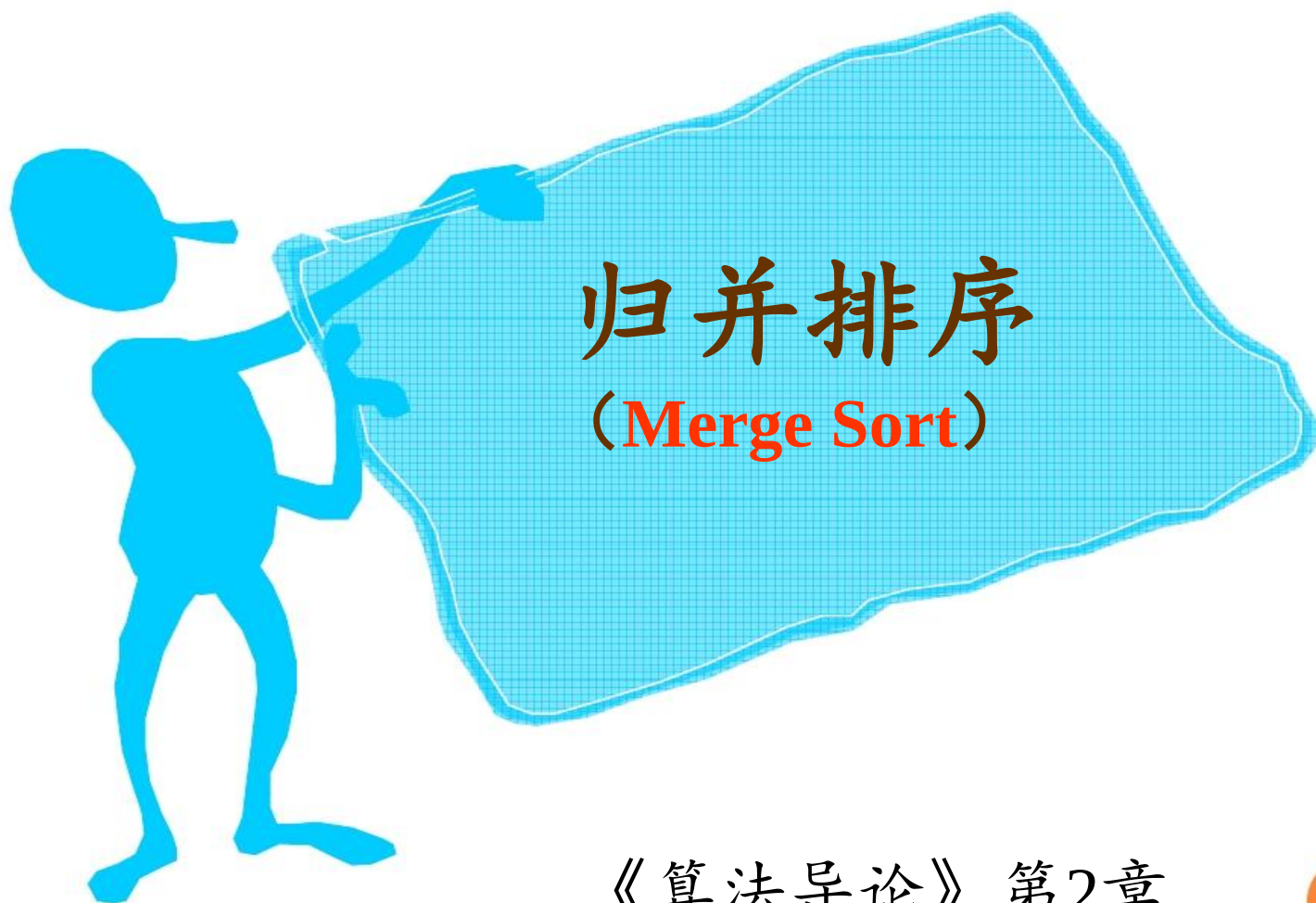
- 划分的策略

1. 选择一个位置将数组划分成两个部分merge-sort
2. 选择一个划分标准 x ，根据元素与 x 的大小关系来划分quick-sort

- 合并策略

不同的划分策略对应不同的合并策略

由于分治思想涉及到递归调用，需要关心子问题的最一般形式，在排序问题中，子问题一般形式就是将 $A[i, \dots, j]$ 中的元素排序



《算法导论》第2章

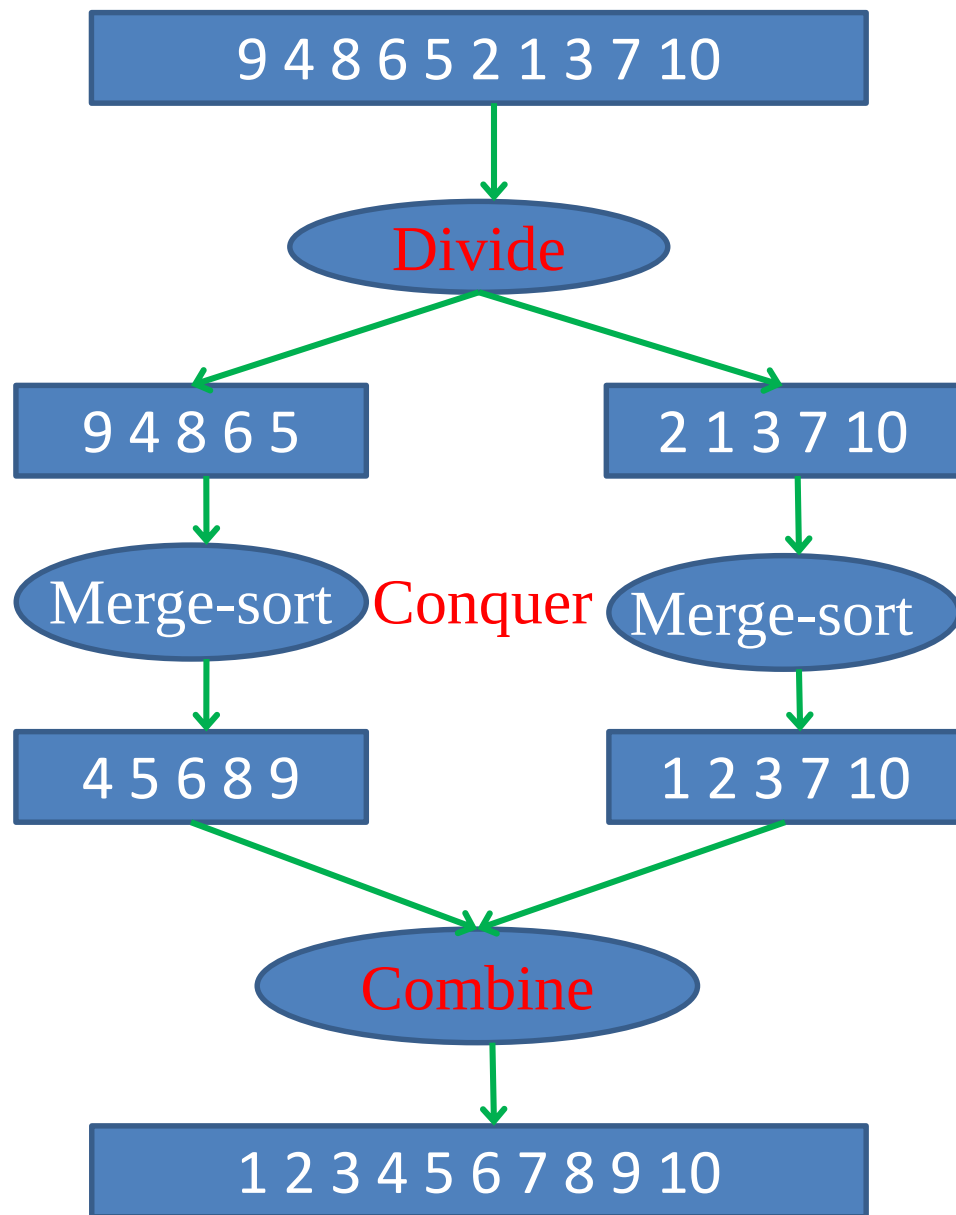


Merge-sort算法

Divide (划分) :

- 需产生划分位置 k
- 第一个子问题是 $A[i, \dots, k]$
- 第二个子问题是 $A[k+1, \dots, j]$
- 为使得两个问题的大小大致相当,
 k 可以如下产生

$$k = \frac{i + j}{2}$$



Merge-sort算法

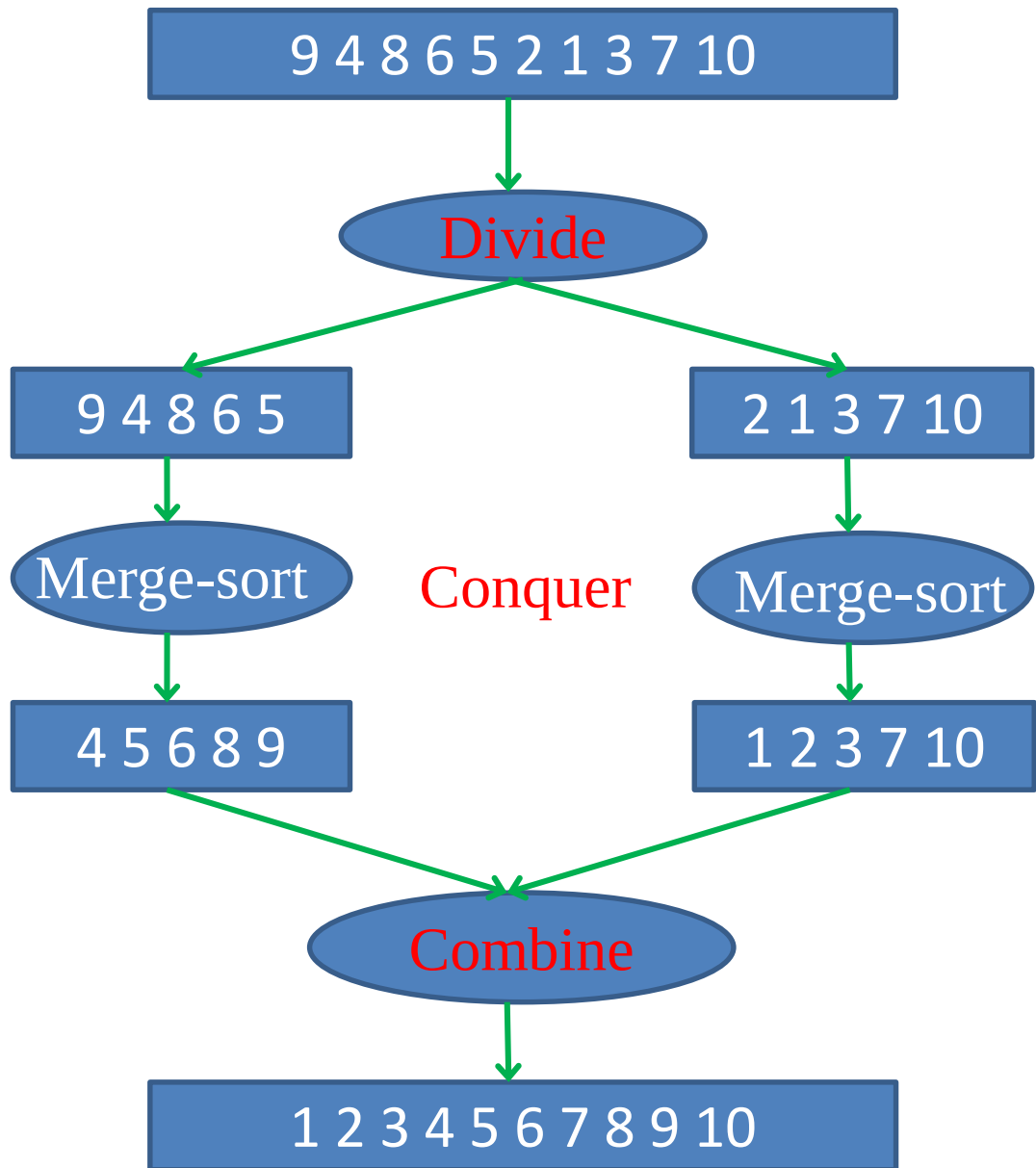
Conquer: 递归求解

//求解第一个子问题

Merge-sort(A, i, k);

//求解第二个子问题

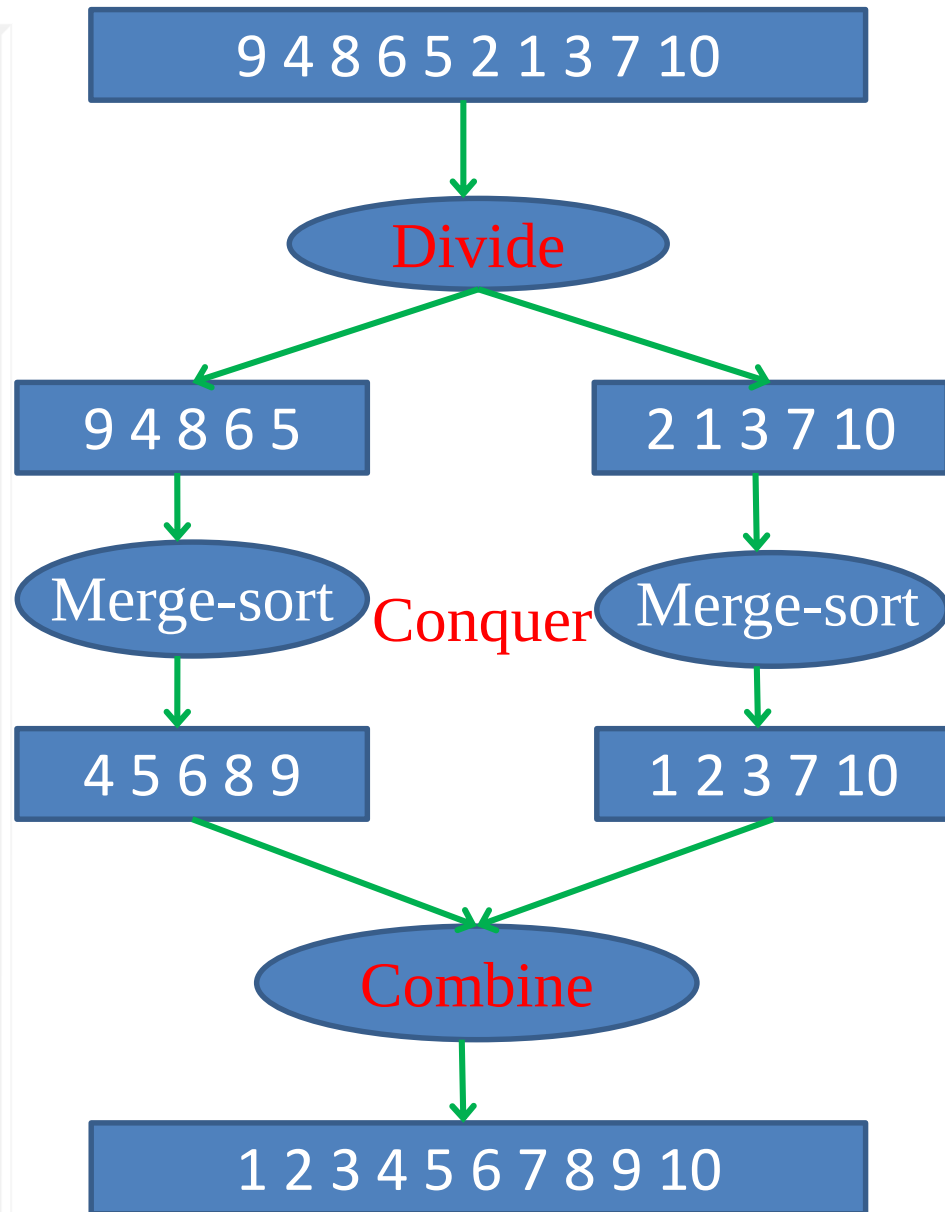
Merge-sort($A, k+1, j$);



Merge-sort算法

Combine(A, i, k, j):%将已排序的子序列合并
 $A[i, \dots, k], A[k+1, \dots, j]$ %

1. $l \leftarrow i; h \leftarrow k+1; t \leftarrow i$ //设置指针
2. While $l \leq k \ \& \ h < j$ Do
6. IF $A[l] < A[h]$ THEN
4. $B[t] \leftarrow A[l]$; //B是排序后的数组
5. $l \leftarrow l + 1$;
6. $t \leftarrow t + 1$;
7. ELSE
8. $B[t] \leftarrow A[h]$;
9. $h \leftarrow h + 1$;
10. $t \leftarrow t + 1$;
11. IF $l < k$ THEN //第一个子问题有剩余元素
12. For $v \leftarrow l$ To k Do
16. $B[t] \leftarrow A[v]$;
14. $t \leftarrow t + 1$;
15. IF $h < j$ THEN //第二个子问题有剩余元素
16. For $v \leftarrow h$ To j Do
17. $B[t] \leftarrow A[v]$;
18. $t \leftarrow t + 1$;
19. For $v \leftarrow i$ To j Do
20. $A[v] \leftarrow B[v]$ //将归并后的数据复制到A中



Merge-sort算法

Merge-sort(**A**,**i**,**j**):

Input: A[i, ..., j]

Output: 排序后的数组A

1. $k \leftarrow (i+j)/2;$

2. Merge-sort(A, i, k); % 递归调用

6. Merge-sort(A, k+1, j);

%将已排序的子序列A[i, ..., k], A[k+1, ..., j]%

4. $l \leftarrow i; h \leftarrow k+1; t \leftarrow i$ //设置指针

5. While $l \leq k$ & $h < j$ Do

6. IF $A[l] < A[h]$ THEN $B[t] \leftarrow A[l]; l \leftarrow l + 1; t \leftarrow t + 1;$

7. ELSE $B[t] \leftarrow A[h]; h \leftarrow h + 1; t \leftarrow t + 1;$

8. IF $l < k$ THEN //第一个子问题有剩余元素

9. For $v \leftarrow l$ To k Do

10. $B[t] \leftarrow A[v]; t \leftarrow t + 1;$

11. IF $h < j$ THEN //第二个子问题有剩余元素

12. For $v \leftarrow h$ To j Do

16. $B[t] \leftarrow A[v]; t \leftarrow t + 1;$

14. For $v \leftarrow i$ To j Do

15. $A[v] \leftarrow B[v]$ //将归并后的数据复制到A中

算法分析

$$D(n) = O(1)$$

$$2T(n/2)$$

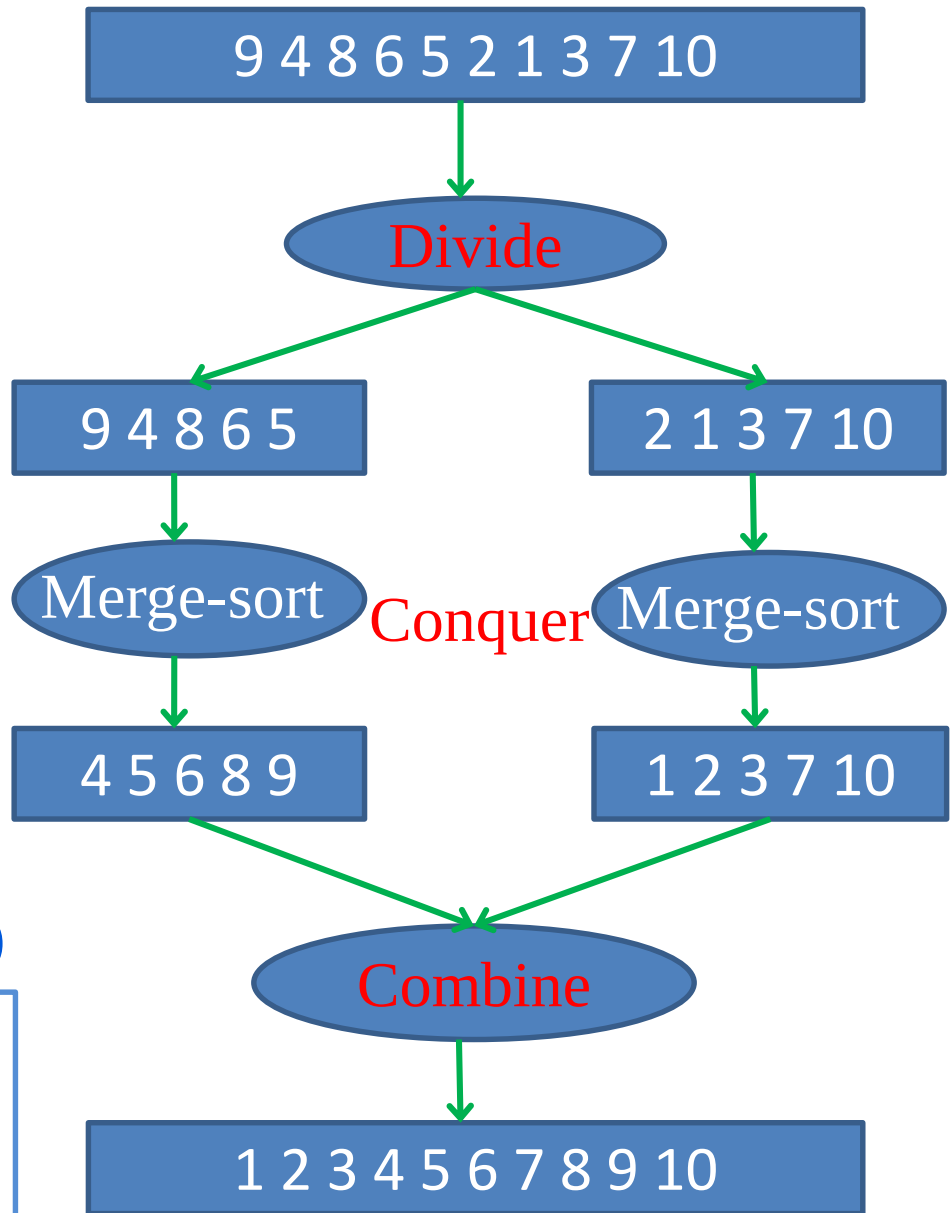
$$C(n) = O(n)$$

建立递归方程

$$\begin{aligned} T(n) &= 2T(n/2) + D(n) + C(n) \\ &= 2T(n/2) + O(n) \end{aligned}$$

使用Master定理

$$T(n) = O(n \lg n)$$



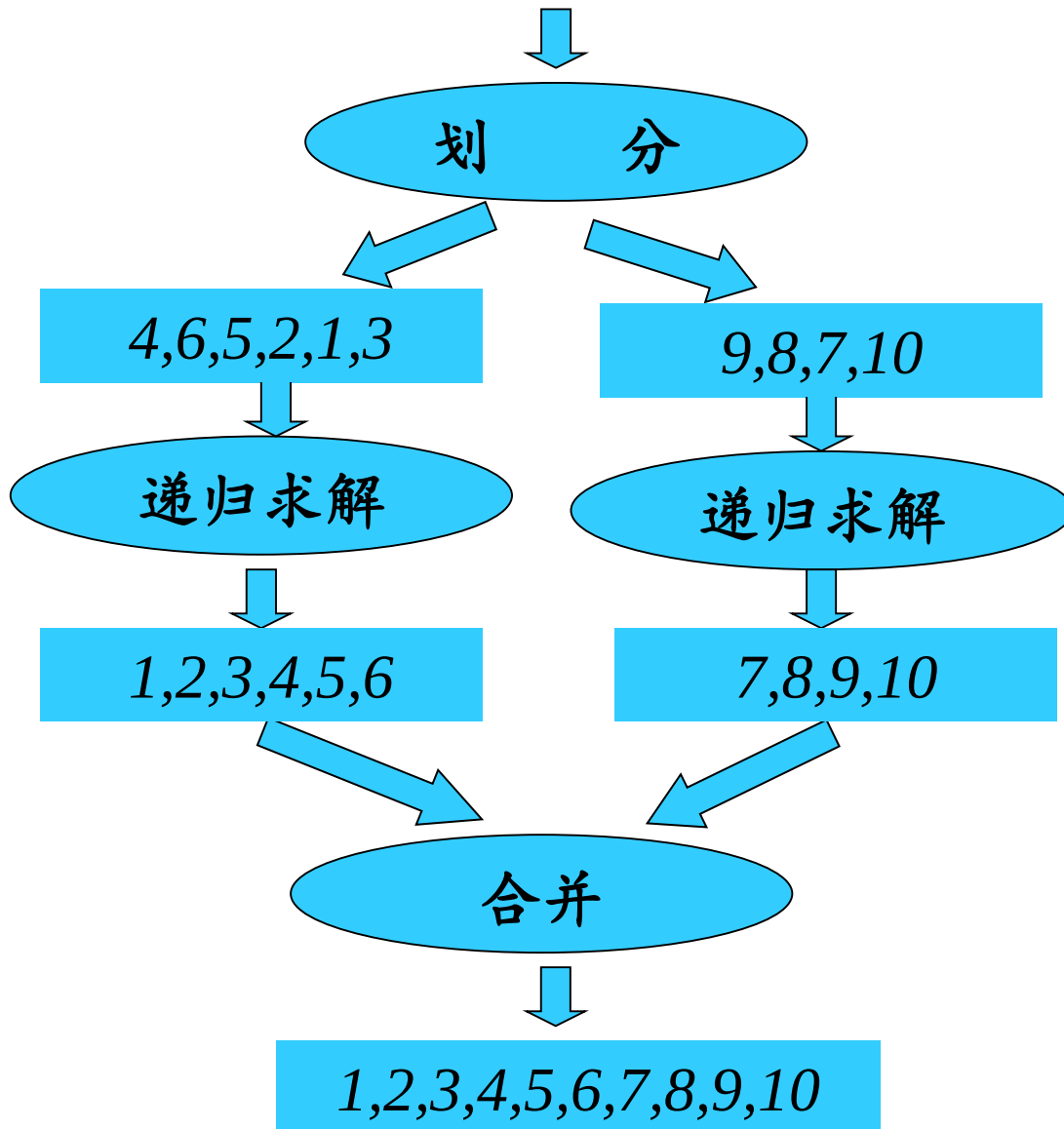


《算法导论》第7章



Quick Sort

9 4 8 6 5 2 1 3 7 10 = $A[i, \dots, j]$



基本思想

Divide:

确定一个划分标准 x ，将数组中小于 x 的元素放到数组的前面部分，大于 x 的元素放到数组的后面部分，并返回一个划分 k ；

Conquer:

递归调用算法将 $A[i, \dots, k]$ 和 $A[k + 1, \dots, j]$ 求解。

Combine:

无操作

QuickSort算法框架

QuickSort(A, i, j):

Input: $A[i, \dots, j], x$

Output: 排序后的数组 $A[i, \dots, j]$

1. 选择划分元素 x ;
 2. $k = \text{Partition}(A, i, j, x);$ // 用 x 完成划分
 6. $\text{QuickSort}(A, i, k);$ // 递归求解子问题
 4. $\text{QuickSort}(A, k+1, j);$ // 递归求解子问题
- // 无需额外的合并操作

Divide

- 指针 low 从低区中找出应放到 x 之后的第一个元素（比 x 大的）
- 指针 $high$ 从高区中找出应放到 x 之前的第一个元素（比 x 小的）
- 如果确实应该交换 low 和 $high$ 标记的两个元素的位置，则交换，否则划分位置就是 $high$

9 4 8 6 5 2 1 3 7 10 = $A[i, \dots, j]$

$x = 7$

9 4 8 6 5 2 1 3 7 10 = $A[i, \dots, j]$

3 4 8 6 5 2 1 9 7 10 = $A[i, \dots, j]$

3 4 1 6 5 2 8 9 7 10 = $A[i, \dots, j]$

3 4 1 6 5 2 8 9 7 10 = $A[i, \dots, j]$

3 4 1 6 5 2 = $A[i, \dots, high]$ 8 9 7 10 = $A[high + 1, \dots, j]$

Divide过程的算法描述

Partition(A, i, j, x):

Input: $A[i, \dots, j], x$

Output: 划分位置k使得 $A[i, \dots, k]$ 中的元素均小于x且 $A[k+1, \dots, j]$ 中的元素均大于等于x

1. $low \leftarrow i; high \leftarrow j;$ //从序列两端交替向中间扫描, 直至两者重叠为止
2. While ($low < high$) Do
6. While ($A[low] < x$) Do
4. $low \leftarrow low + 1;$ //从左向右扫描, 找第1个关键字大于x的 $A[low]$
5. While ($A[high] \geq x$) Do
6. $high \leftarrow high - 1;$ //从右向左扫描, 找第1个关键字小于x的 $A[high]$
7. swap($A[low], A[high]$);
8. return(high)

QuickSort算法

QuickSort(A, i, j):

Input: $A[i, \dots, j], x$

Output: 排序后的数组 $A[i, \dots, j]$

```
1.   $x \leftarrow A[i];$  // 以确定的策略选择 $x$ 
2.   $k = \text{Partition}(A, i, j, x);$  // 用 $x$ 完成划分
6.  QuickSort( $A, i, k$ ); // 递归求解子问题
4.  QuickSort( $A, k+1, j$ );
```

Partition(A, i, j, x):

```
1.   $low \leftarrow i; high \leftarrow j;$ 
2.  While ( $low < high$ ) Do
6.      While ( $A[low] < x$ ) Do
4.           $low \leftarrow low + 1;$  //从左向右扫描, 找第1个关键字大于 $x$ 的 $A[low]$ 
5.      While ( $A[high] \geq x$ ) Do
6.           $high \leftarrow high - 1;$  //从右向左扫描, 找第1个关键字小于 $x$ 的 $A[high]$ 
7.      swap( $A[low], A[high]$ );
8.  return( $high$ )
```

快速排序算法复杂性的分析

最坏情况： $\Theta(n^2)$ （划分不平衡）

对正序或逆序的序列，每次划分只得到一个比上一次划分少一个记录的子序列（另一个子序列为空）。此时，必须经过 $n - 1$ 次递归调用才能把所有记录定位，而且第 i 趟划分需要经过 $n - i$ 次关键码的比较才能找到第 i 个记录的基准位置。

对非正序或逆序的序列，每一轮用最大或者最小元素划分

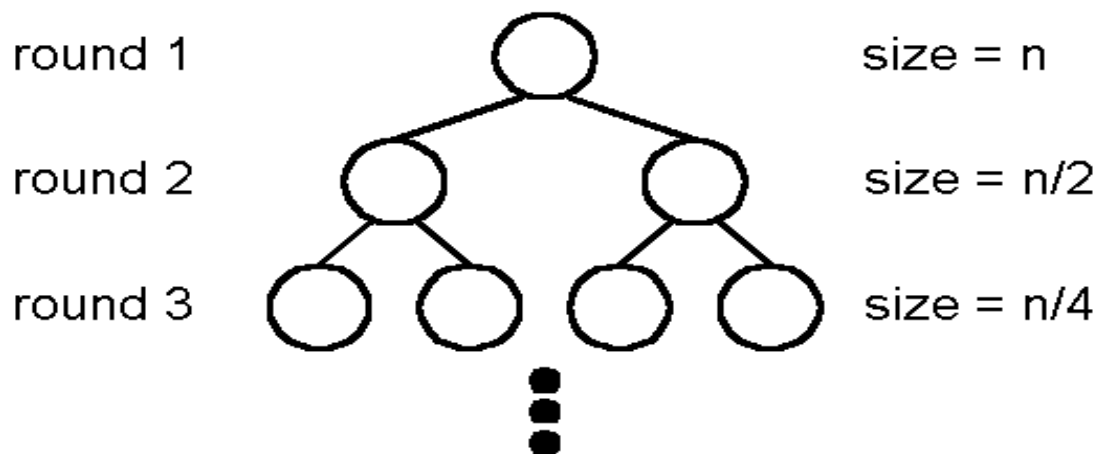
$$\begin{aligned} T(n) &= T(n - 1) + T(0) + \Theta(n) \\ &= T(n - 1) + \Theta(n) \end{aligned}$$

$$(n) + (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2} = \Theta(n^2)$$

快速排序算法复杂性的分析

最好情况： $\Theta(n \lg n)$ （划分尽量平衡）

数组被分为大致相等的两个部分。每次划分对一个记录定位后，该记录的左侧子序列与右侧子序列的长度相同。



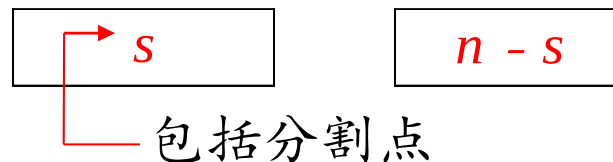
$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

- 需要 $\lg n$ 轮。
- 每轮需要 $O(n)$ 次比较。

快速排序算法复杂性的分析

平均情况： $\Theta(n \lg n)$ 渐进最优的

设划分标准是第 s 小 ($1 \leq s \leq n$)，则有



$$T(n) = \text{Avg}(T(s) + T(n - s))_{1 \leq s \leq n} + cn \quad T(0) = 0$$

$$= \frac{1}{n} \sum_{s=1}^n (T(s) + T(n - s)) + cn$$

$$= \frac{1}{n} (T(1) + T(n - 1) + T(2) + T(n - 2) + \cdots + T(n) + T(0)) + cn$$

$$= \frac{1}{n} (2T(1) + 2T(2) + \cdots + 2T(n - 1) + T(n)) + cn$$

快速排序算法复杂性的分析

$$(n-1)T(n) = 2T(1) + 2T(2) + \cdots + 2T(n-2) + 2T(n-1) + cn^2 \quad (1)$$

$$(n-2)T(n-1) = 2T(1) + 2T(2) + \cdots + 2T(n-2) + c(n-1)^2 \quad (2)$$

$$(1) - (2)$$

$$(n-1)T(n) - (n-2)T(n-1) = 2T(n-1) + c(2n-1) \quad \text{化简和移项}$$

$$(n-1)T(n) - nT(n-1) = c(2n-1) \quad \text{两边同除 } n(n-1)$$

$$\frac{T(n)}{n} = \frac{T(n-1)}{n-1} + c\left(\frac{1}{n} + \frac{1}{n-1}\right)$$

$$\frac{T(n-1)}{n-1} = \frac{T(n-2)}{n-2} + c\left(\frac{1}{n-1} + \frac{1}{n-2}\right)$$

.....

$$\frac{T(2)}{2} = \frac{T(1)}{1} + c\left(\frac{1}{2} + 1\right)$$

快速排序算法复杂性的分析

$$\frac{T(n)}{n} = \frac{T(n-1)}{n-1} + c\left(\frac{1}{n} + \frac{1}{n-1}\right)$$

$$\frac{T(n-1)}{n-1} = \frac{T(n-2)}{n-2} + c\left(\frac{1}{n-1} + \frac{1}{n-2}\right)$$

.....

$$\frac{T(2)}{2} = \frac{T(1)}{1} + c\left(\frac{1}{2} + 1\right)$$

$$\frac{T(n)}{n} = c\left(\frac{1}{n} + \frac{1}{n-1}\right) + c\left(\frac{1}{n-1} + \frac{1}{n-2}\right) + \cdots + c\left(\frac{1}{2} + 1\right) + T(1)$$

$$= c\left(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{2}\right) + c\left(\frac{1}{n-1} + \frac{1}{n-2} + \cdots + 1\right)$$

$$H_{n-1}$$

快速排序算法复杂性的分析

调和级数 [Kunth 1986]

$$\begin{aligned} H_n &= 1 + \frac{1}{2} + \cdots + \frac{1}{n-1} + \frac{1}{n} \\ &= \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + \frac{1}{120n^4} - \varepsilon, \end{aligned}$$

其中 $0 < \varepsilon < \frac{1}{252n^6}$

$\gamma = 0.5772156649 \dots$

$$H_n = O(\lg n)$$

$$\begin{aligned} \frac{T(n)}{n} &= c \left(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{2} \right) + c \left(\frac{1}{n-1} + \frac{1}{n-2} + \cdots + 1 \right) \\ &= c(H_n - 1) + cH_{n-1} \\ &= c \left(2H_n - \frac{1}{n} - 1 \right) \\ \Rightarrow T(n) &= 2cnH_n - c(n+1) = O(n \lg n) \end{aligned}$$



《算法导论》第7章



随机QuickSort算法

- 快速排序算法的性能取决于划分的对称性。通过修改算法 **partition**，可以设计出采用随机选择策略的快速排序算法。在快速排序算法的每一步中，当数组还没有被划分时，可以在 $A[i:j]$ 中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以期望划分是较对称的。

随机QuickSort算法

RandomQuickSort(A, i, j):

Input: $A[i, \dots, j], x$

Output: 排序后的数组 $A[i, \dots, j]$

1. $temp \leftarrow rand(i, j)$

// 产生 i, j 之间的随机数

2. $x \leftarrow A[temp];$

// 以确定的策略选择 x

2. $k = Partition(A, i, j, x);$

// 用 x 完成划分

6. QuickSort(A, i, k);

// 递归求解子问题

4. QuickSort($A, k+1, j$);

Partition(A, i, j, x):

1. $low \leftarrow i; high \leftarrow j;$

2. While ($low < high$) Do

6. While ($A[low] < x$) Do

4. $low \leftarrow low + 1;$

5. While ($A[high] \geq x$) Do

6. $high \leftarrow high - 1;$

7. swap($A[low], A[high]$);

8. return($high$);

算法性能的分析

• 基本概念

- $S_{(i)}$ 表示 S 中第 i 小的元素

例如, $S_{(1)}$ 和 $S_{(n)}$ 分别是最小和最大元素

- 随机变量 X_{ij} 定义如下:

$X_{ij} = 1$, 如果 $S_{(i)}$ 和 $S_{(j)}$ 在运行中被比较, 否则为 0

- X_{ij} 是 $S_{(i)}$ 和 $S_{(j)}$ 的比较次数

- 算法的比较次数为
$$\sum_{i=1}^n \sum_{j>i} X_{ij}$$

- 算法的平均复杂性为
$$E \left[\sum_{i=1}^n \sum_{j>i} X_{ij} \right] = \sum_{i=1}^n \sum_{j>i} E[X_{ij}]$$

- 计算 $E[X_{ij}]$

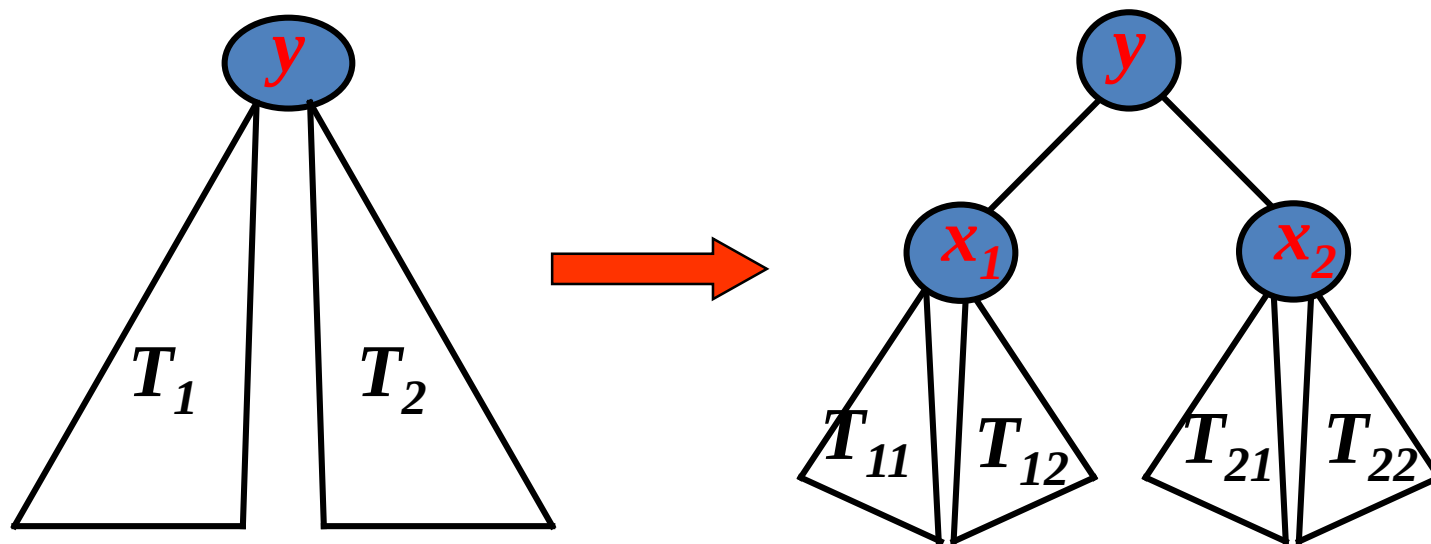
- 设 p_{ij} 为 $S_{(i)}$ 和 $S_{(j)}$ 在运行中比较的概率, 则

$$E[X_{ij}] = p_{ij} \times 1 + (1 - p_{ij}) \times 0 = p_{ij}$$

关键问题成为求解 p_{ij}

- 求解 P_{ij}

- 我们可以用树表示算法的计算过程



- 我们可以观察到如下事实：

- 一个子树的根必须与其子树的所有节点比较
- 不同子树中的节点不可能比较
- 任意两个节点至多比较一次

- 当主元 x 满足 $S_{(i)} < x < S_{(j)}$ ，那么 $S_{(i)}$ 和 $S_{(j)}$ 将不会被比较。例如2和8
- 当 $S_{(i)}, S_{(i+1)}, \dots, S_{(j)}$ 在同一子树时， $S_{(i)}$ 和 $S_{(j)}$ 才可能比较
- 只有 $S_{(i)}$ 或 $S_{(j)}$ 先于 $S_{(i+1)}, S_{(i+2)}, \dots, S_{(j-1)}$ 被选为划分点时， $S_{(i)}$ 和 $S_{(j)}$ 才可能比较。例如7和9
- $S_{(i)}, S_{(i+1)}, \dots, S_{(j)}$ 等可能地被选为划分点，所以 $S_{(i)}$ 和 $S_{(j)}$

进行比较的概率是： $\frac{2}{j-i+1}$ ，即

$$p_{ij} = \frac{2}{j-i+1}$$

课本P102页-103页

- 现在我们有

$$\begin{aligned} \sum_{i=1}^n \sum_{j>i} E[X_{ij}] &= \sum_{i=1}^n \sum_{j>i} p_{ij} = \sum_{i=1}^n \sum_{j>i} \frac{2}{j-i+1} \\ &\leq \sum_{i=1}^n \sum_{k=1}^{n-i+1} \frac{2}{k} = 2 \sum_{i=1}^n \sum_{k=1}^{n-i+1} \frac{1}{k} = 2nH_n = O(n \lg n) \text{调和级数} \end{aligned}$$

定理.

随机快速排序算法的期望时间复杂度为 $O(n \lg n)$ 。



排序问题的下界

- 问题的下界是解决该问题的算法所需要的最小时间复杂性。
 - ☆ 最坏情况下界
 - ☆ 平均情况下界
- 排序问题的下界是不唯一的
 - 例如. $\Omega(n), \Omega(n \lg n)$ 都是排序的下界

- 如果一个问题的下界是 $\Omega(n \lg n)$ ，而当前最好算法的时间复杂度是 $O(n^2)$ 。
 - 我们可以寻找一个更高的下界。
 - 我们可以设计更好的算法。
 - 下界和算法都是可以改进的。
- 如果一个问题的下界是 $\Omega(n \lg n)$ 且算法的时间复杂度是 $O(n \lg n)$ ，那么这个算法是最优的。

最坏情况下排序的下界

3个元素有6种排列

a_1	a_2	a_3
1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

1,2,3指的是元素下标

直接插入排序

- 输入: ($a_1 = 5, a_2 = 6, a_3 = 4$)

(1) $a_1 : a_2$

(2) $a_2 : a_3, a_2 \leftrightarrow a_3$

(3) $a_1 : a_2, a_1 \leftrightarrow a_2$

- 输入: ($a_1 = 5, a_2 = 4, a_3 = 6$)

(1) $a_1 : a_2, a_1 \leftrightarrow a_2$

(2) $a_2 : a_3$

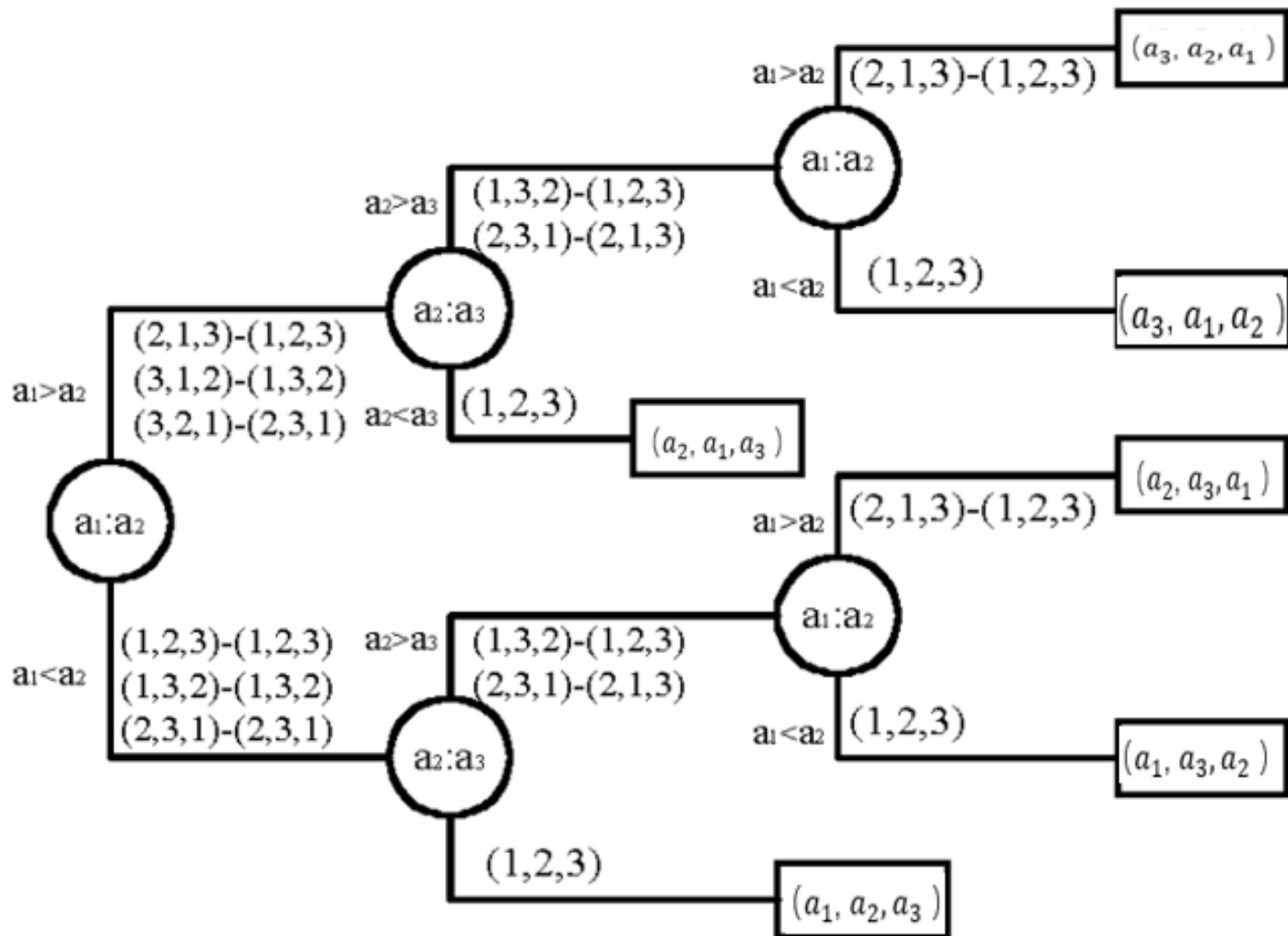
Insertion-sort(A)

Input: $A[1, \dots, n] = n$ 个数

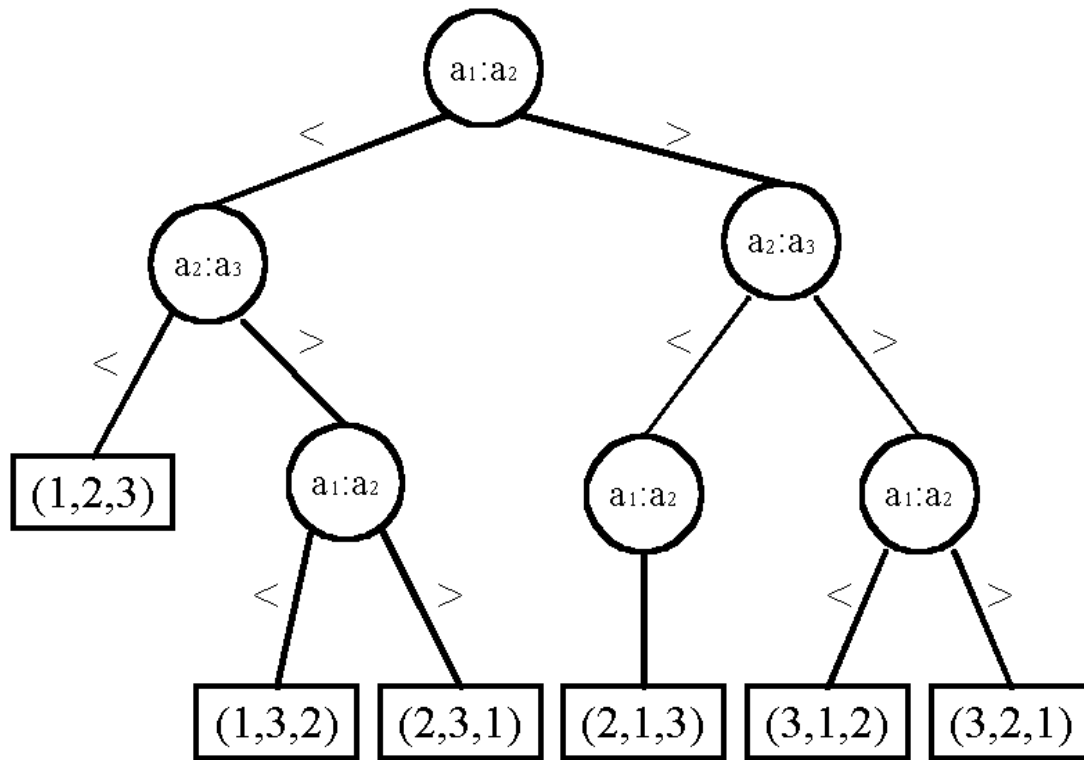
Output: $A[1, \dots, n] = n$ 个sorted数

```
{1. For j = 2 To n Do
  2. key ← A[j];
  3. %将A[j] 插入到A[1, ..., j-1]%
  4. i ← j-1;
  5. While i > 0 and A[i] > key Do
  6. { A[i+1] ← A[i];
  7.   i ← i-1; }
  8. A[i+1] ← key;
}
```

直接插入排序的决策树



冒泡排序的决策树



(1) 比较排序都可以抽象成一棵决策树，且为完全二叉树。

(2) 排序算法的实际执行对应于一条从树的根结点到叶结点的路径。即每个叶结点表示一个正确的排序序列。

(3) 每个内部结点表示一次比较。

(4) n 个元素有 $n!$ 种不同排列，且都出现在决策树的叶结点上。

排序的下界

- n 个元素有 $n!$ 种不同排列，那么二叉决策树有 $n!$ 个叶子结点。
- 为了找到排序的下界，我们需要找到二叉树的最小高度。
- 排序的下界对应于平衡树最小高度

考虑一颗高度为 h 、具有 l 个可达叶节点的决策树，它对应一个对 n 个元素所做的比较排序。因为输入数据的 $n!$ 种不同排列都是叶节点，所以有 $n! \leq l$ 。由于在一棵高为 h 的二叉树中，叶节点数据不多于 2^h ，那么有

$$n! \leq l \leq 2^h$$

两边取对数，得

$$h \geq \lg(n!) = \Omega(n \lg n)$$

排序的下界是： $\Omega(n \lg n)$

方法1:

$$\lg(n!) = \lg(1 \times 2 \times \cdots \times (n-1) \times n)$$

$$= \lg(2) + \lg(3) + \cdots + \lg(n)$$

$$= \sum_{k=2}^n \lg(k)$$

$$> \int_{2-1}^n \lg(x) dx$$

$$= \lg e \int_1^n \ln(x) dx$$

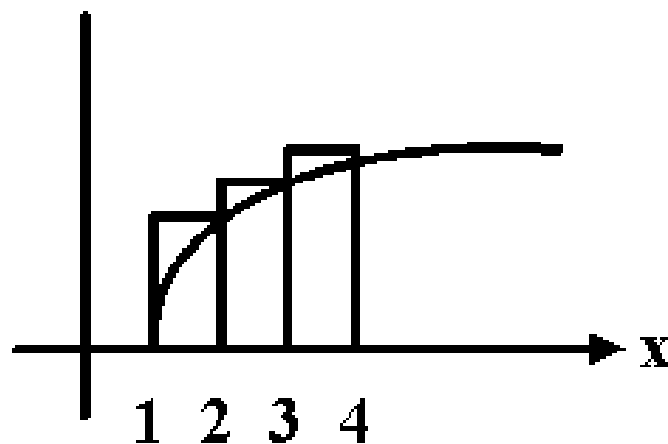
$$= \lg e [x \ln(x) - x]_1^n$$

$$= \lg e (n \ln(n) - n + 1)$$

$$= \lg e * n \lg(n) - \lg e * n + \lg e$$

$$\geq n \lg(n) - 1.44 * n$$

$$= \Omega(n \lg n)$$



如果 $f(k)$ 单调递增, 则 $\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx$ 。

方法2:

Stirling（斯特林）近似：（是一条用来取 n 的阶乘的近似值的数学公式）

$$- n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

$$- \lg(n!) \approx \lg \sqrt{2\pi} + \frac{1}{2} \lg n + n \lg \frac{n}{e} \approx n \lg n = \Omega(n \lg n)$$

n	n!	S_n
1	1	0.922
2	2	1.919
3	6	5.825
4	24	23.447
5	120	118.02
6	720	707.39
10	3,628,800	3,598,600
20	2.433×10^{18}	2.423×10^{18}
100	9.333×10^{157}	9.328×10^{157}

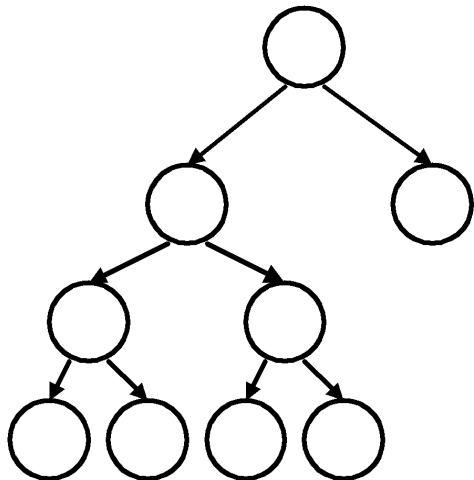
排序的平均情况下界

- 仍利用决策树
- 排序算法的平均复杂性用从根结点到每个叶子结点的路径长度的平均长度描述。

$$\frac{\text{总长度}}{n!}$$

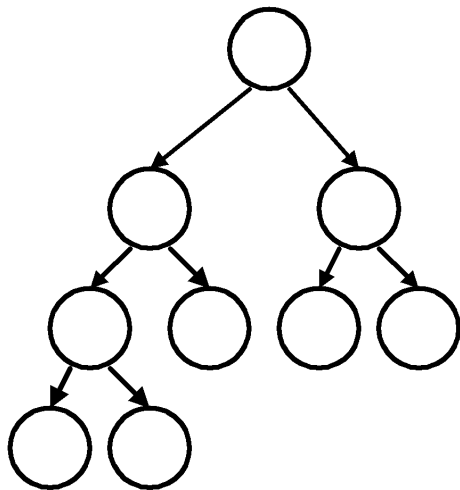
- 当树平衡时这个值最小.
(所有叶子结点的深度是 d 或 $d-1$)

平均情况下排序的下界



不平衡的情况下
总长度

$$= 4 \cdot 3 + 1 = 13$$



平衡的情况下
总长度

$$= 2 \cdot 3 + 3 \cdot 2 = 12$$

计算最小路径长度和

1. 有 c 个叶子结点的二叉树的深度 $d = \lceil \lg c \rceil$

叶子结点仅出现在 d 或 $d-1$ 层

2. x_1 个结点在 $d-1$ 层

x_2 个结点在 d 层

$$\blacksquare x_1 + x_2 = c$$

$$\blacksquare x_1 + \frac{x_2}{2} = 2^{d-1}$$

$$\Rightarrow \begin{aligned} x_1 &= 2^d - c \\ x_2 &= 2(c - 2^{d-1}) \end{aligned}$$

6. 总长度

$$\begin{aligned}M &= x_1(d-1) + x_2d \\&= (2^d - c)(d-1) + 2(c - 2^{d-1})d \\&= c(d-1) + 2(c - 2^{d-1}), \quad d-1 = \lfloor \lg c \rfloor \\&= c\lfloor \lg c \rfloor + 2(c - 2^{\lfloor \lg c \rfloor})\end{aligned}$$

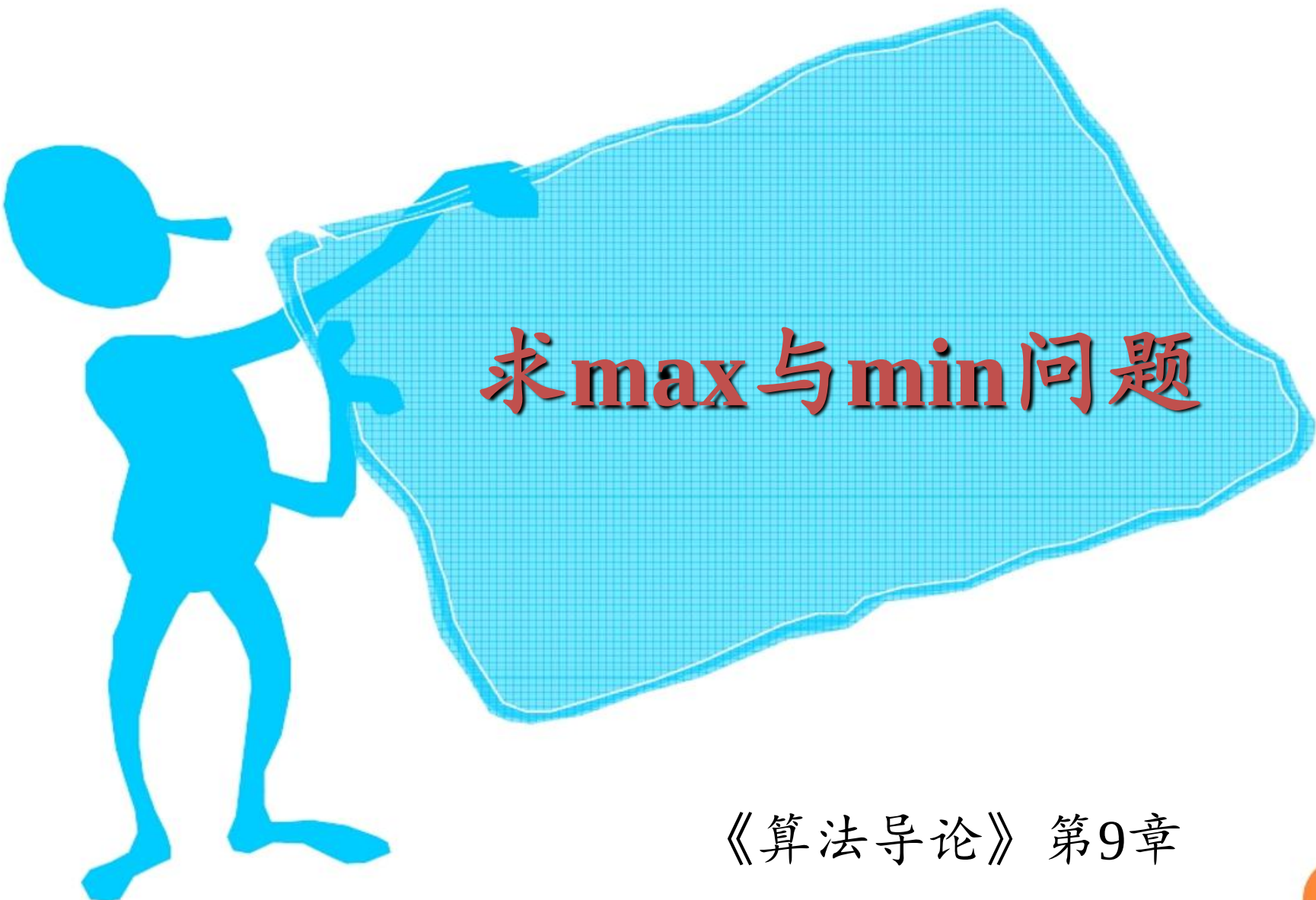
4. $c = n!$ 因为 n 个元素有 $n!$ 种不同排列, 且都出现在决策树的叶结点上。

$$\begin{aligned}M &= n! \lfloor \lg n! \rfloor + 2(n! - 2^{\lfloor \lg n! \rfloor}) \\M/n! &= \lfloor \lg n! \rfloor + 2 \\&= \lfloor \lg n! \rfloor + c, \quad 0 \leq c \leq 1 \\&= \Omega(n \lg n)\end{aligned}$$

平均情况下排序的下界是: $\Omega(n \lg n)$

总结

- 排序的下界是： $\Omega(n \lg n)$
- 平均情况下排序的下界是： $\Omega(n \lg n)$
- 在最坏情况下，任何比较排序算法都需要做 $\Omega(n \lg n)$ 次比较。
- 快速排序在相差一个常数因子的范围内平均是最优的。



求max与min问题

《算法导论》第9章



选择问题

输入： n 个（互异）数构成的集合 $A[1, \dots, n]$ ，
整数 i ， $1 \leq i \leq n$

输出： 元素 $x \in A$ ， 且 A 中恰好有 $i - 1$ 个其他元素小于它

解决方法：

可以采用归并排序和快速排序对 A 排序，
然后输出第 i 个元素。

复杂度 $\Omega(n \lg n)$ 。

➤ 最大值和最小值问题

➤ 中位数问题

线性时间选择

- 本节讨论如何在 $O(n)$ 时间内从 n 个不同的数中选取第 i 大的元素
- 这个问题解决了，最大值和最小值问题 和中位数问题也就解决了

求max与min问题 问题定义

输入：数组 $A[1, \dots, n]$ ，整数 $i = 1$ 和 $i = n$

输出： A 中的最大值和最小值

通常，直接扫描需要 $2n - 2$ 次比较操作

我们给出一个仅需 $\left\lceil \frac{3n}{2} - 2 \right\rceil$ 次比较操作的算法。

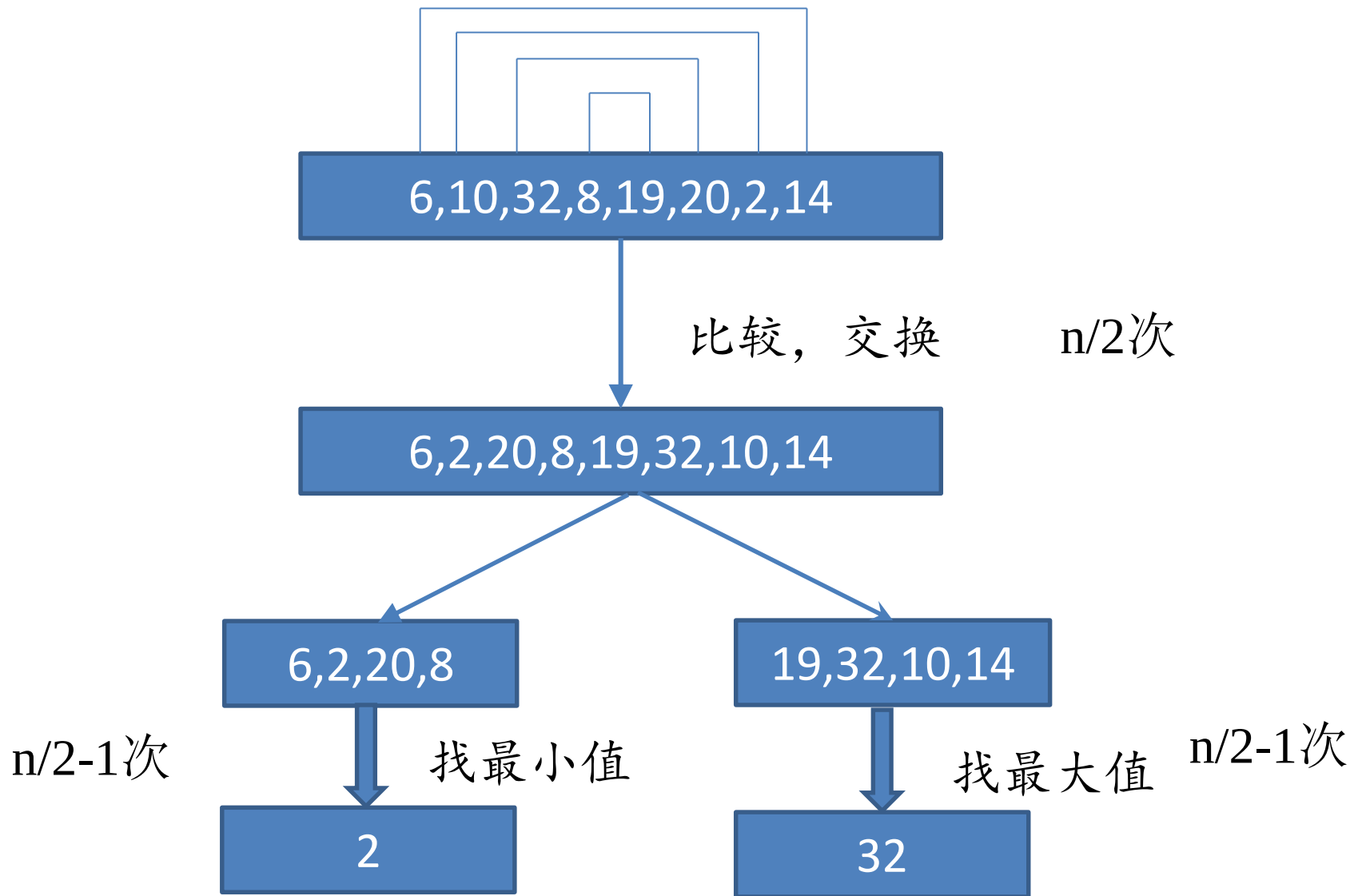
基本思想

基于任务不同将问题划分成两个子问题

一个子问题求解最大值，另一个子问题求解最小值

- 1、将 $A[i]$ 与 $A[n - i + 1]$ 比较， $i = 1, 2, \dots, n/2$
- 2、较小的元素放在前面，较大的元素放在后面
- 3、最小元素出现在 $A[1, 2, \dots, [n/2]]$ 中
- 4、最大元素 $A[[n/2], \dots, n]$ 中

算法 1



算法 1

Max-min(A)

Input: 数组 $A[1, \dots, n]$

Output: 数组 $A[1, \dots, n]$ 中的 max 和 min

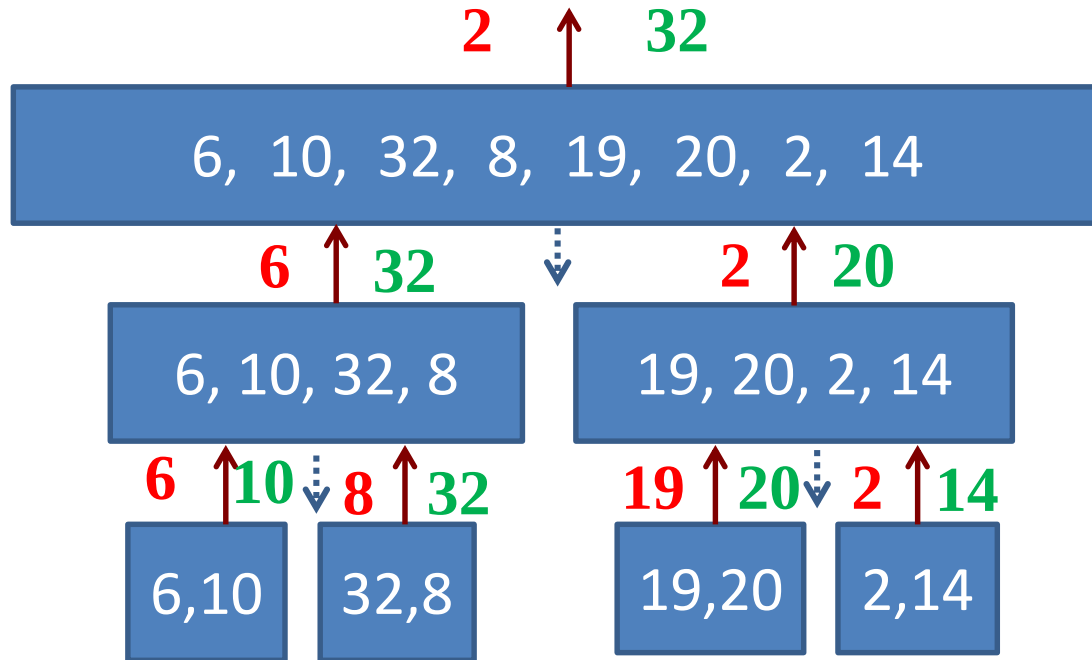
```
{1.  For  $i \leftarrow 1$  To  $n/2$  Do
    2.  IF  $A[i] > A[n-i+1]$  THEN swap( $A[i], A[n-i+1]$ );
    6.   $\max \leftarrow A[n]$ ;  $\min \leftarrow A[1]$ ;
    4.  For  $i \leftarrow 2$  To  $\lceil n/2 \rceil$  Do
    5.  IF  $A[i] < \min$  THEN  $\min \leftarrow A[i]$ ;
    6.  IF  $A[n-i+1] > \max$  THEN  $\max \leftarrow A[n-i+1]$ ;
    7.  print  $\max, \min$ ;
}
```

比较次数: $\left\lceil \frac{3n}{2} - 2 \right\rceil$ 次比较操作

这个方法是分治法吗?

算法 2

红色字体：最小值
绿色字体：最大值



算法 2

Max-min(A)

Input: 数组 $A[1, \dots, n]$

Output: (x, y) , $A[1, \dots, n]$ 中的 max 和 min

Max-min(A, 1, n)

过程: **Max-min(A, low, high)**

```
{1. IF high - low = 1
2.   IF  $A[\text{low}] < A[\text{high}]$  THEN return( $A[\text{low}], A[\text{high}]$ );
6.   ELSE return( $A[\text{high}], A[\text{low}]$ );
4. ELSE
5.    $\text{mid} \leftarrow (\text{low} + \text{high}) / 2$ ;
6.    $(x_1, y_1) \leftarrow \text{Max-min}(A, \text{low}, \text{mid})$ 
7.    $(x_2, y_2) \leftarrow \text{Max-min}(A, \text{mid} + 1, \text{high})$ 
8.    $x \leftarrow \min\{x_1, x_2\}$ 
9.    $y \leftarrow \max\{y_1, y_2\}$ 
10. return( $x, y$ );
}
```

$$T(1) = 0 \quad (\text{不要比较大小})$$

$$T(2) = 1 \quad (\text{比较大小1次})$$

$$T(n) = 2T\left(\frac{n}{2}\right) + 2$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2^2 + 2$$

$$= \dots$$

$$= 2^{k-1} T(2) + 2^{k-1} + \dots + 2^2 + 2 \quad n = 2^k$$

$$= 3 \frac{n}{2} - 2$$

1、为什么“+2”？

因为两个子序列需要比较出较大和较小元素。

2、不再研究最大最小值选择问题的下界和平均情况下选择的下界。



元素选取问题



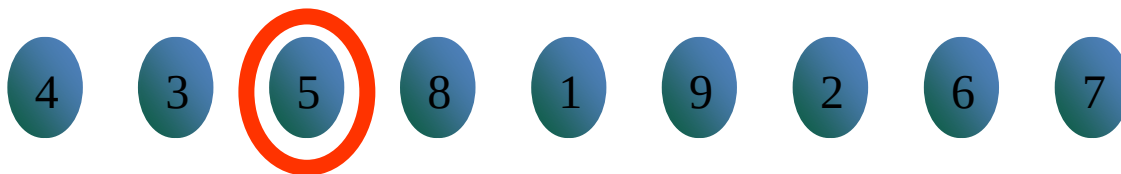
中位数问题定义

输入：由 n 个数构成的多重集合 X

输出： $x \in X$ 使得 $-1 \leq |\{y \in X | y < x\}| - |\{y \in X | y > x\}| \leq 1$

输入：多重集合 $X[1, \dots, n]$ ，整数 $i = \left\lfloor \frac{n+1}{2} \right\rfloor$

输出： A 的中位数 x



中位数选取问题的复杂度



- [Blum et al. STOC'72 & JCSS'73]
 - A “shining” paper by five authors:
 - Manuel Blum (Turing Award 1995)
 - Robert W. Floyd (Turing Award 1978)
 - Vaughan R. Pratt
 - Ronald L. Rivest (Turing Award 2002)
 - Robert E. Tarjan (Turing Award 1986)
 - 从 n 个数中选取中位数需要的比较操作的次数介于 $1.5n$ 到 $5.43n$ 之间

线性时间选择

- 上界

- $3n + o(n)$ by Schonhage, Paterson, and Pippenger (JCSS 1975)
- $2.95n$ by Dor and Zwick (SODA 1995, SIAM Journal on Computing 1999)

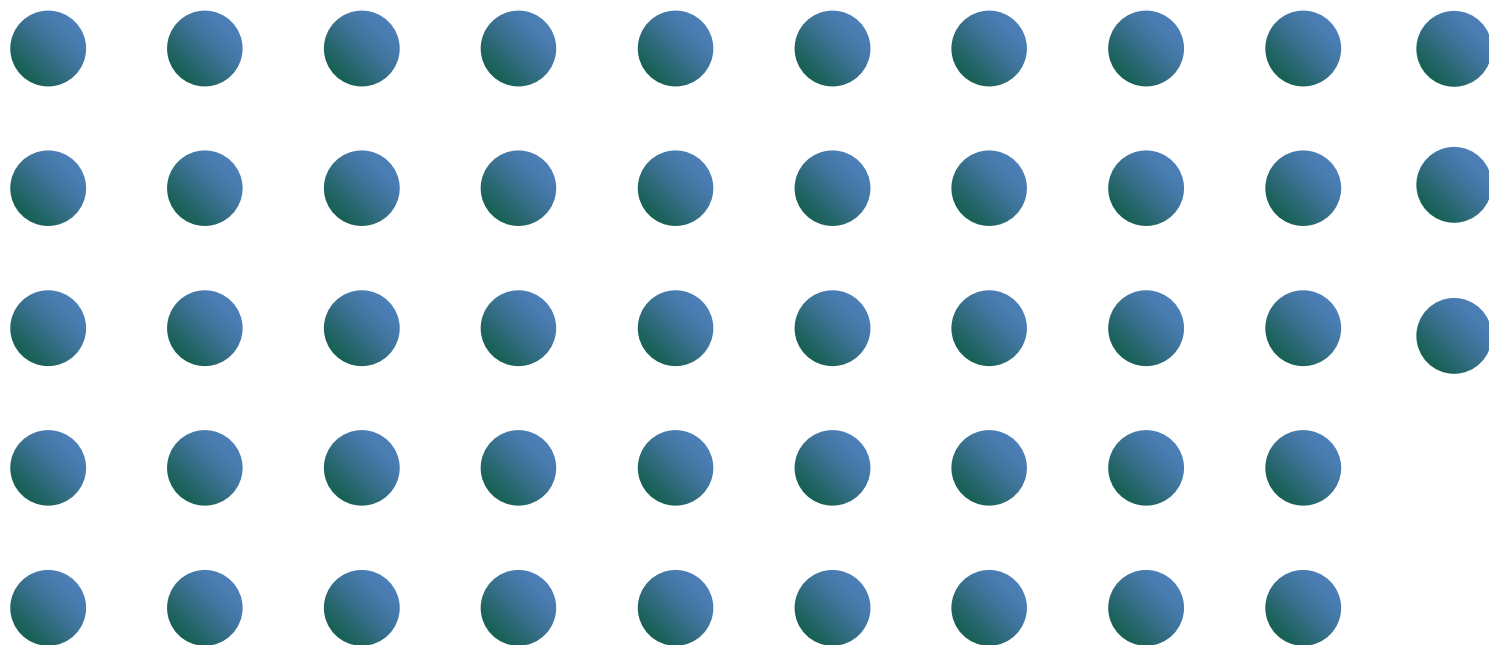
- 下界

- $2n + o(n)$ by Bent and John (STOC 1985)
- $(2 + 2^{-80})n$ by Dor and Zwick (FOCS 1996, SIAM Journal on Discrete Math 2001)

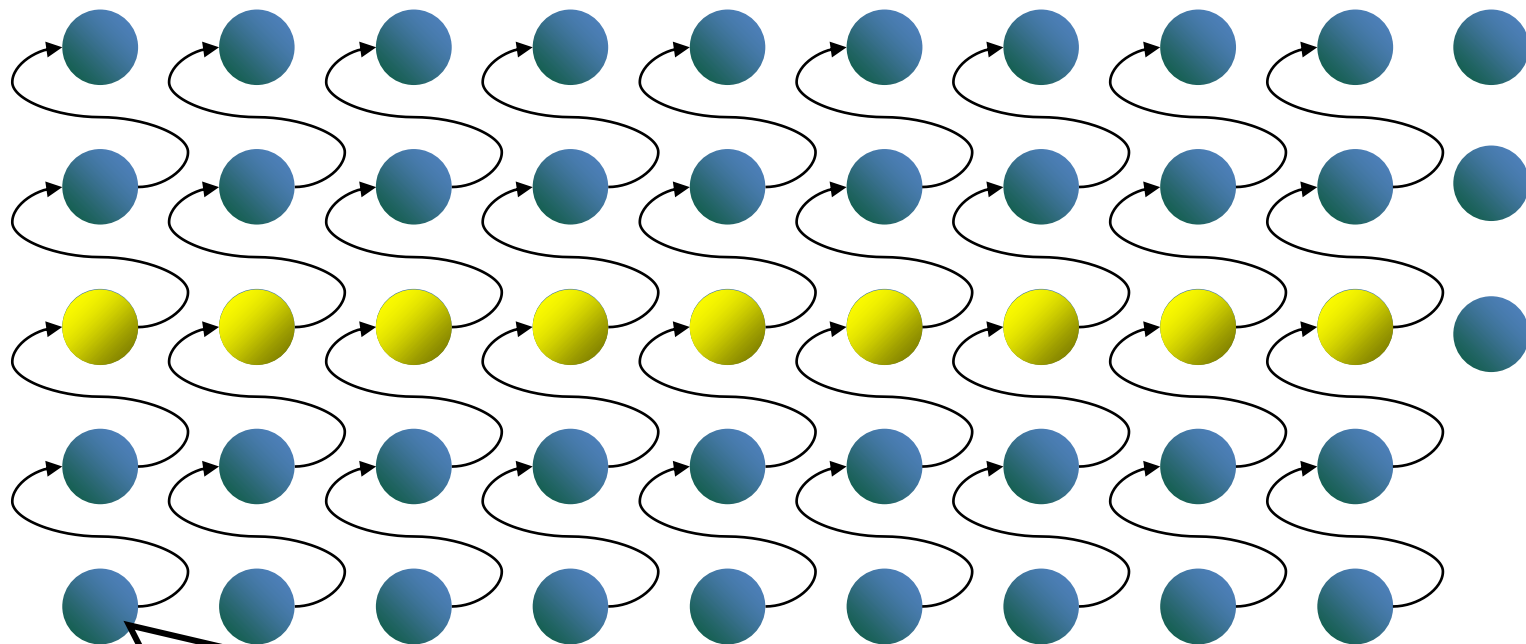
求解步骤（分治法）

第一步： 分组，每组5个数

最后一组可能少于5个数

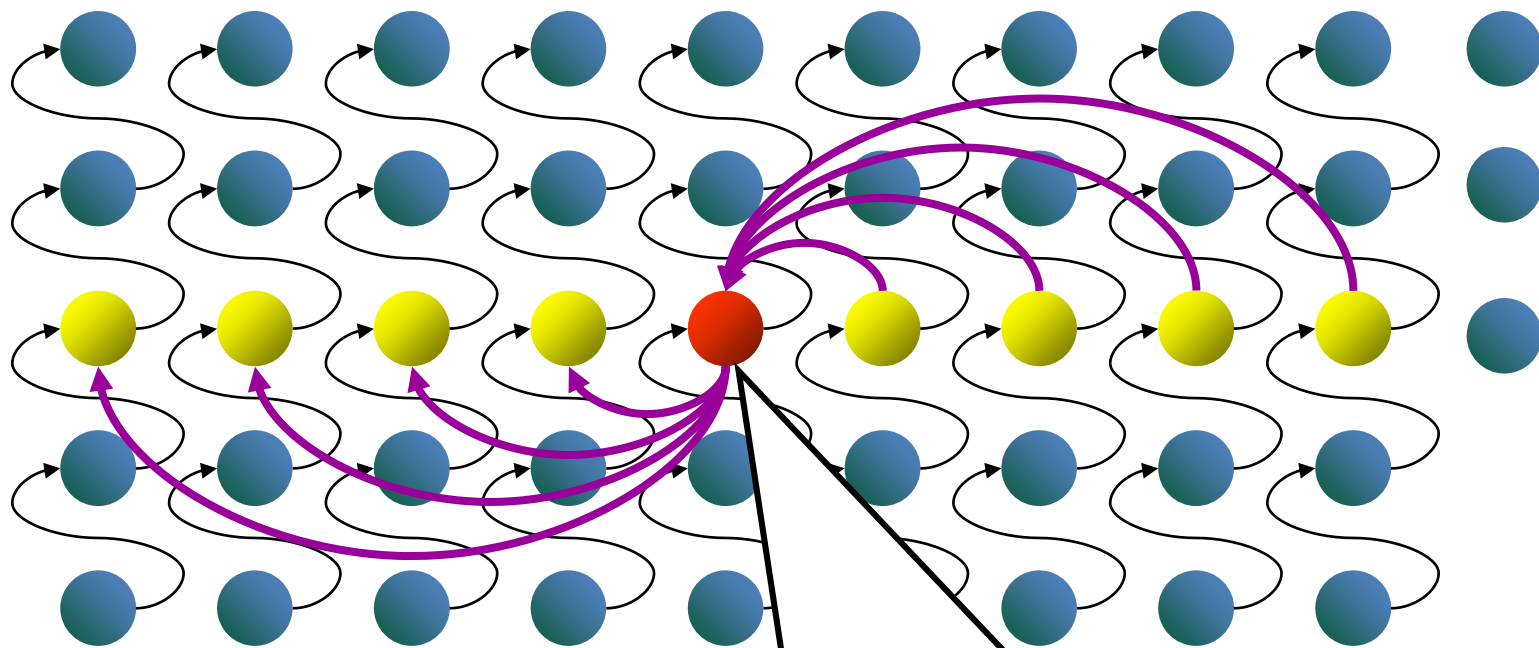


第二步：将每组数分别用InsertionSort插入排序 选出每组元素的中位数



排序每组数时，比较操作的次数最坏情况下为 $\frac{5(5-1)}{2} = 10$ 次
总共需要 $10 * \lfloor n/5 \rfloor$ 次比较操作，其中 $\lfloor n/5 \rfloor$ 是组数

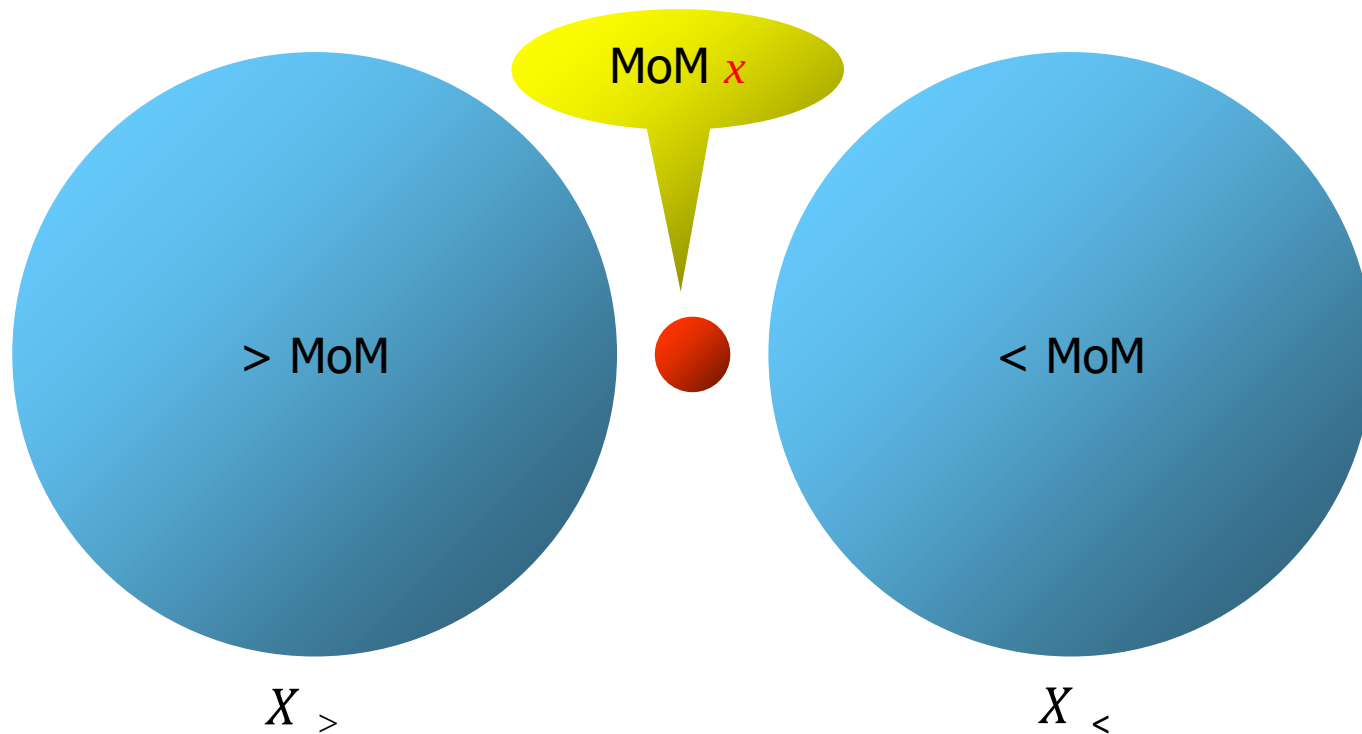
第三步：递归调用算法求得这些中位数的中位数（MoM）



时间复杂性： $T(\lfloor n/5 \rfloor)$ 。

原始问题规模是 n ，现在问题规模是 $\lfloor n/5 \rfloor$

第四步：用MoM完成划分



时间复杂性 $O(n)$

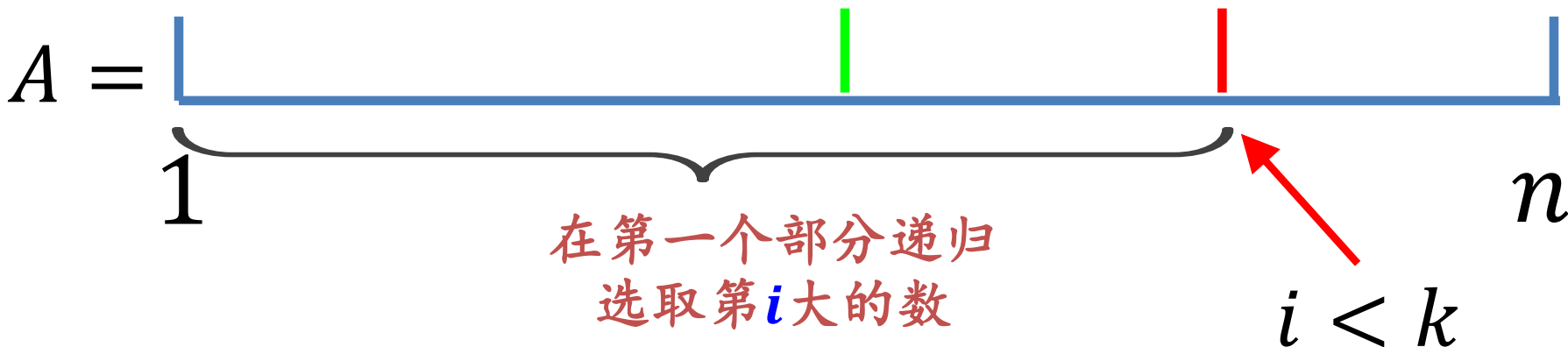
第五步:递归

设 x 是中位数的中位数 (MoM)，划分完成后 x 的下标为 k

- 如果 $i = k$ ，则返回 x
- 如果 $i < k$ ，则在第一个部分递归选取第 i 大的数

如果 $i < k$ ，说明找的中位数过头了，太大了，这时候只要在 $1 \sim k$ 之间选第 i 大的数

$$i = \left\lfloor \frac{n+1}{2} \right\rfloor$$

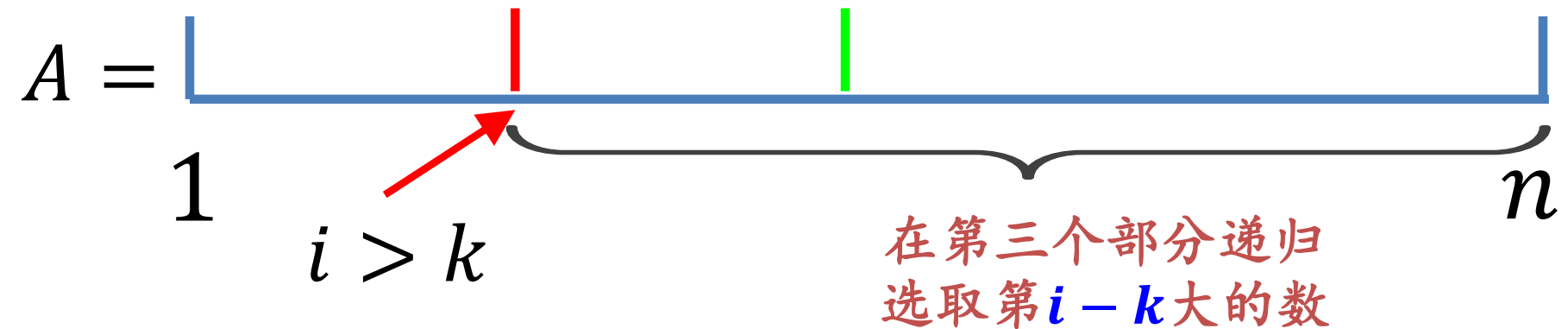


第五步:递归

设 x 是中位数的中位数 (MoM)，划分完成后 x 的下标为 k

- 如果 $i = k$ ，则返回 x
- 如果 $i < k$ ，则在第一个部分递归选取第 i 大的数
- 如果 $i > k$ ，则在第三个部分递归选取第 $i - k$ 大的数

$$i = \left\lfloor \frac{n+1}{2} \right\rfloor$$



算法Select(A, i):

Input: 数组A[1, ..., n], $1 \leq i \leq n$

Output: A[1, ..., n]中的第i-大的数

1. For $j \leftarrow 1$ to $n/5$

2. Insertion-sort(A[(j-1)*5+1:j*5]);

6. Swap(B[j], A[(j-1)*5+3]);

4. $x \leftarrow$ Select(B[1:n/5], n/10);

5. $k \leftarrow$ partition(A[1:n], x);

6. IF $k == i$ THEN return x;

7. ELSE IF $k > i$ THEN return Select(A[1:k-1], i);

8. ELSE
 return Select(A[k+1:n], i-k);

← 第一步

} 第二步

← 第三步

← 第四步

} 第五步

算法分析

算法Select(A, i):

Input: 数组A[1, ..., n], $1 \leq i \leq n$

Output: A[1, ..., n]中的第i-大的数

1. For $j \leftarrow 1$ to $n/5$

2. InsertSort(A[(j-1)*5+1:(j-1)*5+5]);

6. Swap(B[j], A[(j-1)*5+3]);

4. $x \leftarrow \text{Select}(B[1:n/5], n/10);$

5. $k \leftarrow \text{partition}(A[1:n], x);$

6. IF $k = i$ THEN return x ;

7. ELSE IF $k > i$ THEN return $\text{Select}(A[1:k-1], i);$

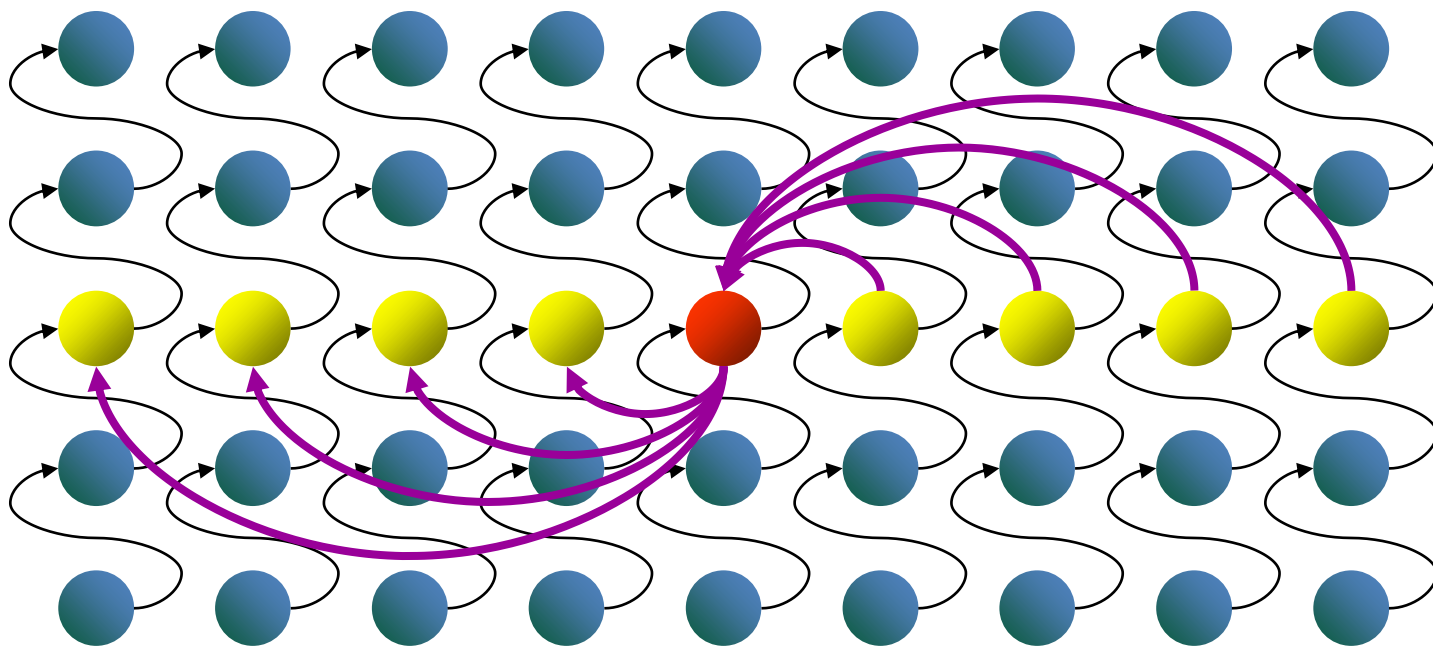
8. ELSE return $\text{Select}(A[k+1:n], i-k);$

} $O(n)$
 $T(\lfloor n/5 \rfloor)$

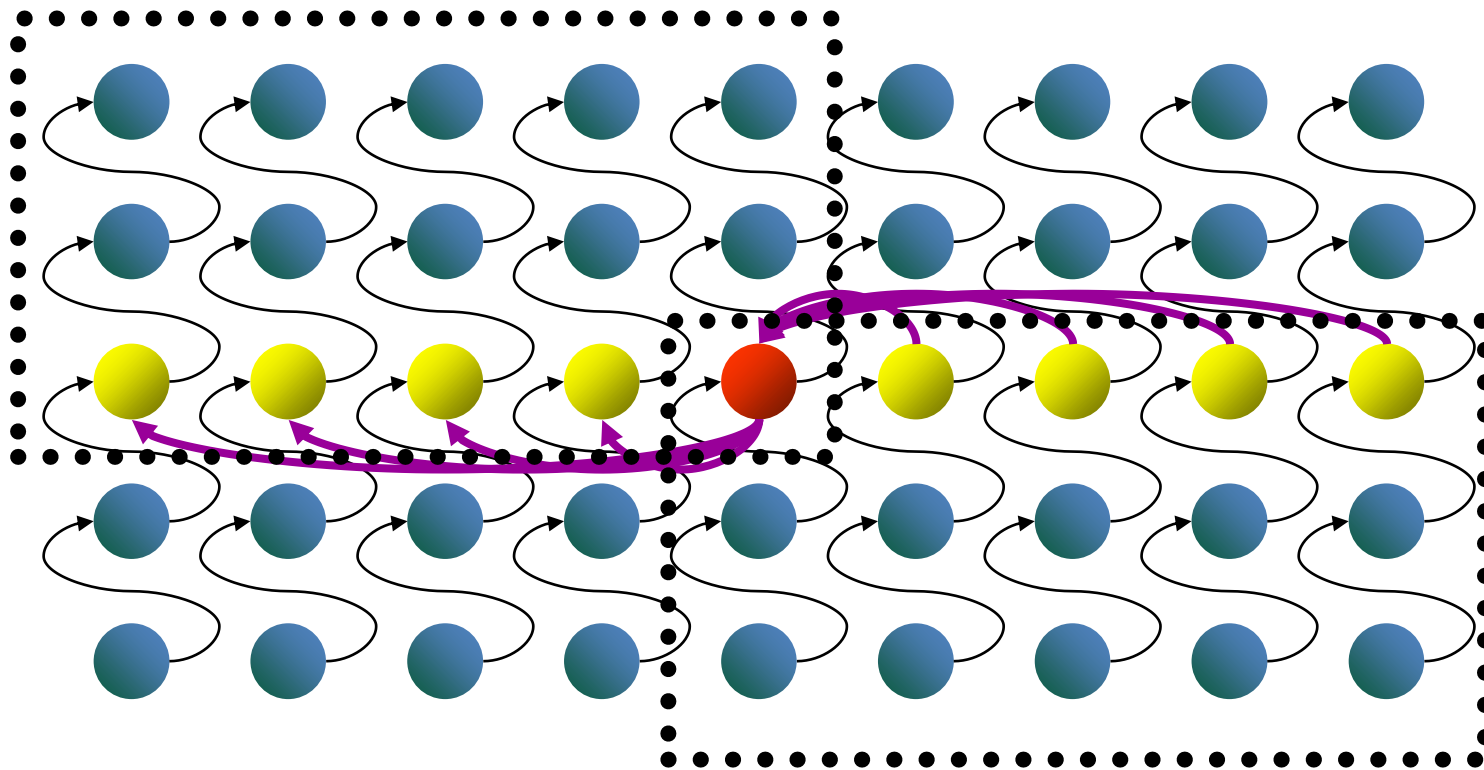
← $O(n)$

} ????

观察第五步的处理过程



第五步至少删除了 $\lfloor 3n/10 \rfloor$ 个数



第五步至少要删除虚线框中的元素，如果 $i < k$ ，那么第二个虚线框中的元素可以删除，前面的虚线框中元素和左下角和右上角的都要保留，因为和 k 的大小情况不知道

$$n - \lfloor 3^n / 10 \rfloor \leq 7^n / 10 + 6$$

如果时间复杂度是输入规模的递增函数

则第五步的时间开销不超过 $T(7n/10 + 6)$

这些元素
是多少个?
原来 $n/5$ 组，
取一半，
是 $n/10$ ，然
后每组里
面取3个，
就是 $3n/10$

算法分析

建立递归方程：

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq C \\ T\left(\left\lfloor \frac{n}{5} \right\rfloor\right) + T\left(\frac{7n}{10} + 6\right) + \Theta(n) & \text{if } n > C \end{cases}$$

时间复杂度：

$$T(n) = O(n)$$

大数乘法



大数乘法 问题定义

输入： n 位二进制整数 X 和 Y

输出： X 和 Y 的乘积

通常计算时间复杂度是 $O(n^2)$, 我们接下来给出一个复杂度是 $O(n^{\lg 3}) = O(n^{1.59})$ 的算法

简单分治算法

$$X = \overset{n/2\text{位}}{A} \overset{n/2\text{位}}{B} \quad Y = \overset{n/2\text{位}}{C} \overset{n/2\text{位}}{D}$$

$$\begin{aligned} XY &= (A \times 2^{n/2} + B)(C \times 2^{n/2} + D) \\ &= AC \times 2^n + (AD + BC) \times 2^{n/2} + BD \end{aligned}$$

算法

1. 计算 A, B, C, D ;
2. 计算 $n/2$ 位乘法 AC 、 AD 、 BC 、 BD ;
3. 计算 $AD + BC$;
4. AC 左移 n 位, $(AD + BC)$ 左移 $n/2$ 位;
5. 计算 XY

算法复杂性:

$$T(n) = 4T(n/2) + \Theta(n), \text{ 使用Master定理: } T(n) = O(n^2)$$

如何降低复杂度呢? 减少子问题的数量

改进分治算法

$$X = \overset{n/2\text{位}}{\boxed{A}} \overset{n/2\text{位}}{\boxed{B}} \quad Y = \overset{n/2\text{位}}{\boxed{C}} \overset{n/2\text{位}}{\boxed{D}}$$

$$\begin{aligned} XY &= (A \times 2^{n/2} + B)(C \times 2^{n/2} + D) \\ &= AC \times 2^n + (AD + BC) \times 2^{n/2} + BD \\ &= AC \times 2^n + ((A - B)(D - C) + AC + BD) \times 2^{n/2} + BD \end{aligned}$$

算法

1. 计算 A, B, C, D ;
2. 计算 $A - B$ 和 $D - C$;
3. 计算 $n/2$ 位乘法 AC 、 BD 、 $(A - B)(D - C)$;
4. 计算 $(A - B)(D - C) + AC + BD$;
5. AC 左移 n 位, $((A - B)(D - C) + AC + BD)$ 左移 $n/2$ 位;
6. 计算 XY

算法复杂性:

$$T(n) = 3T(n/2) + \Theta(n), \text{ 使用 Master 定理: } T(n) = O(n^{\lg 3})$$

算法分析

建立递归方程：

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1 \end{cases}$$

使用Master定理：

$$T(n) = O(n^{\lg 3}) = O(n^{1.59})$$

能否利用此思想去解决大规模矩阵乘法问题？

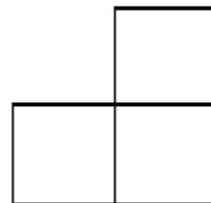
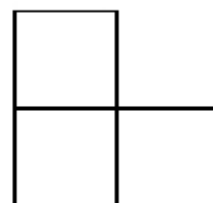
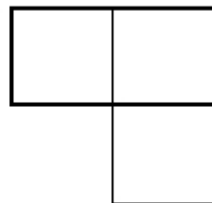
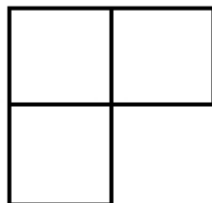
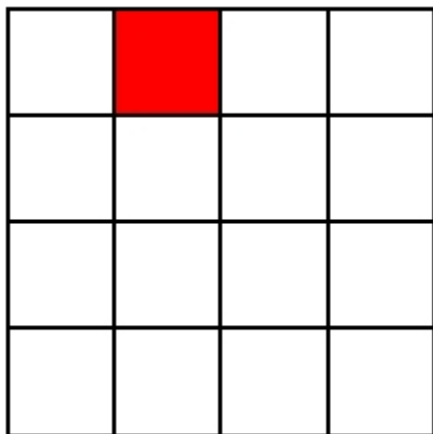


棋盘覆盖问题

棋盘覆盖问题 问题定义

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，有一个方格与其它的方格不同，称之为奇异块。

要求：若使用以下四种L型骨牌覆盖除这个奇异块的其它方格，覆盖过程中L型骨牌间不能有互相覆盖，设计算法求出覆盖方案。四个L型骨牌如下图：



分治思想

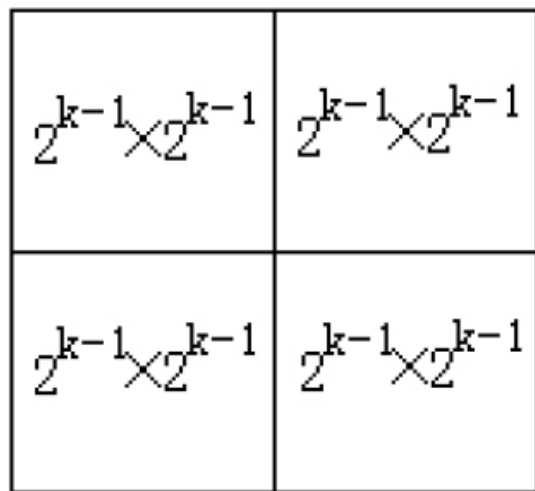
1、当 $k > 0$ 时，将 $2^k \times 2^k$ 的棋盘分成四块 $2^{(k-1)} \times 2^{(k-1)}$ 的子棋盘

特殊方格一定在其中的一个子棋盘中，其他相邻的三个子棋盘中没有特殊方格

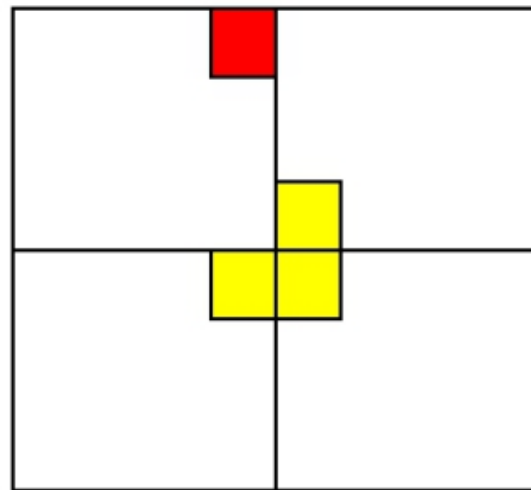
2、使用一个合适的L型骨牌，覆盖三个没有特殊方格的子棋盘的相邻方格

我们就得到了4个 $2^{(k-1)} \times 2^{(k-1)}$ 的棋盘覆盖问题

3、继续递归处理4子棋盘，直到子棋盘中只有一个特殊方格为止

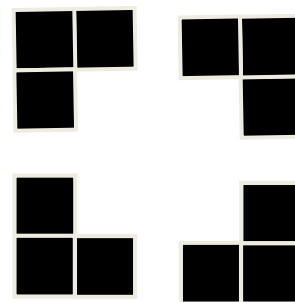
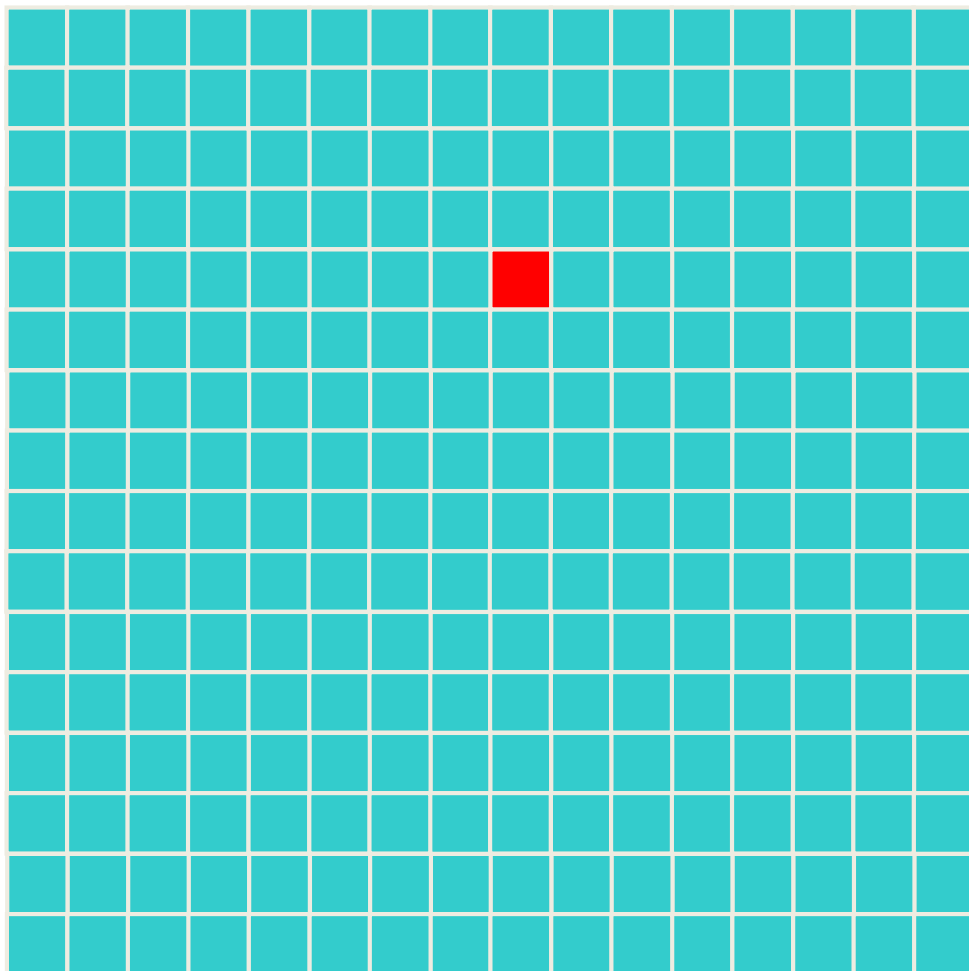


(a)

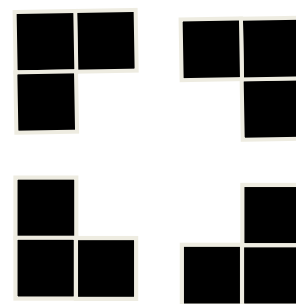
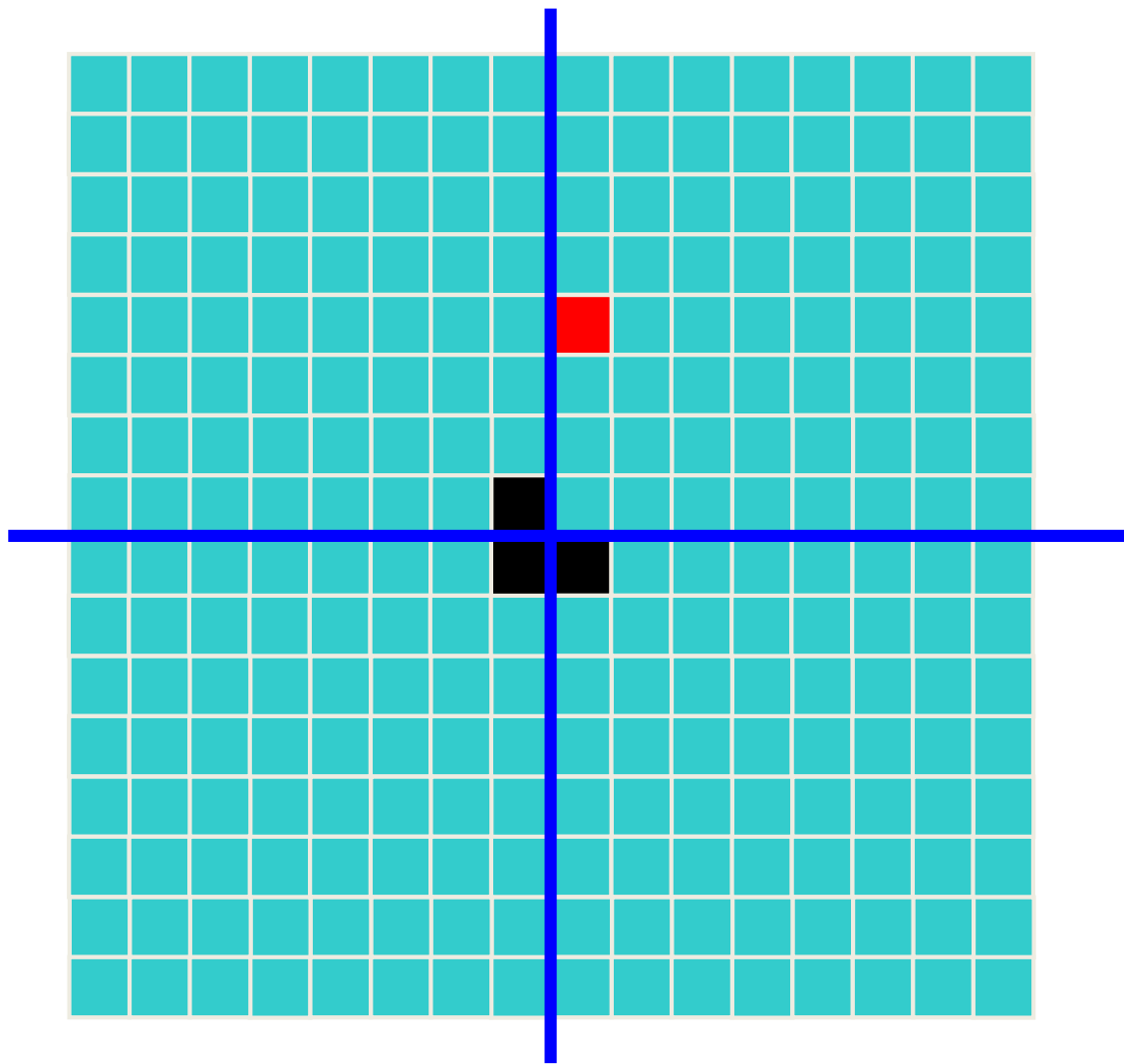


(b)

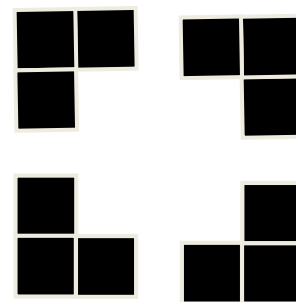
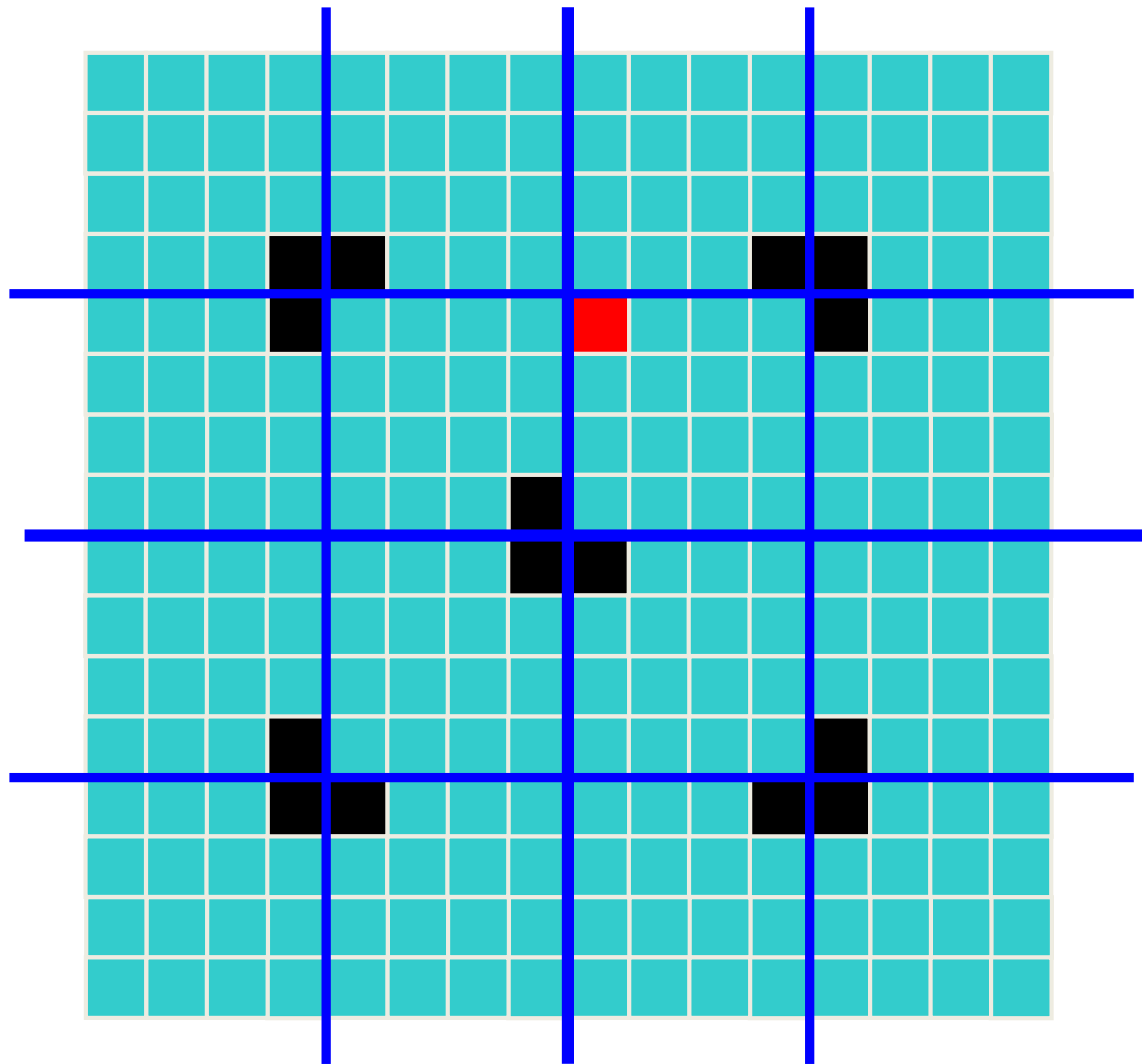
详细过程图解



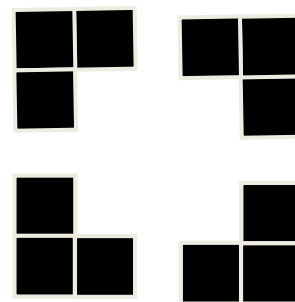
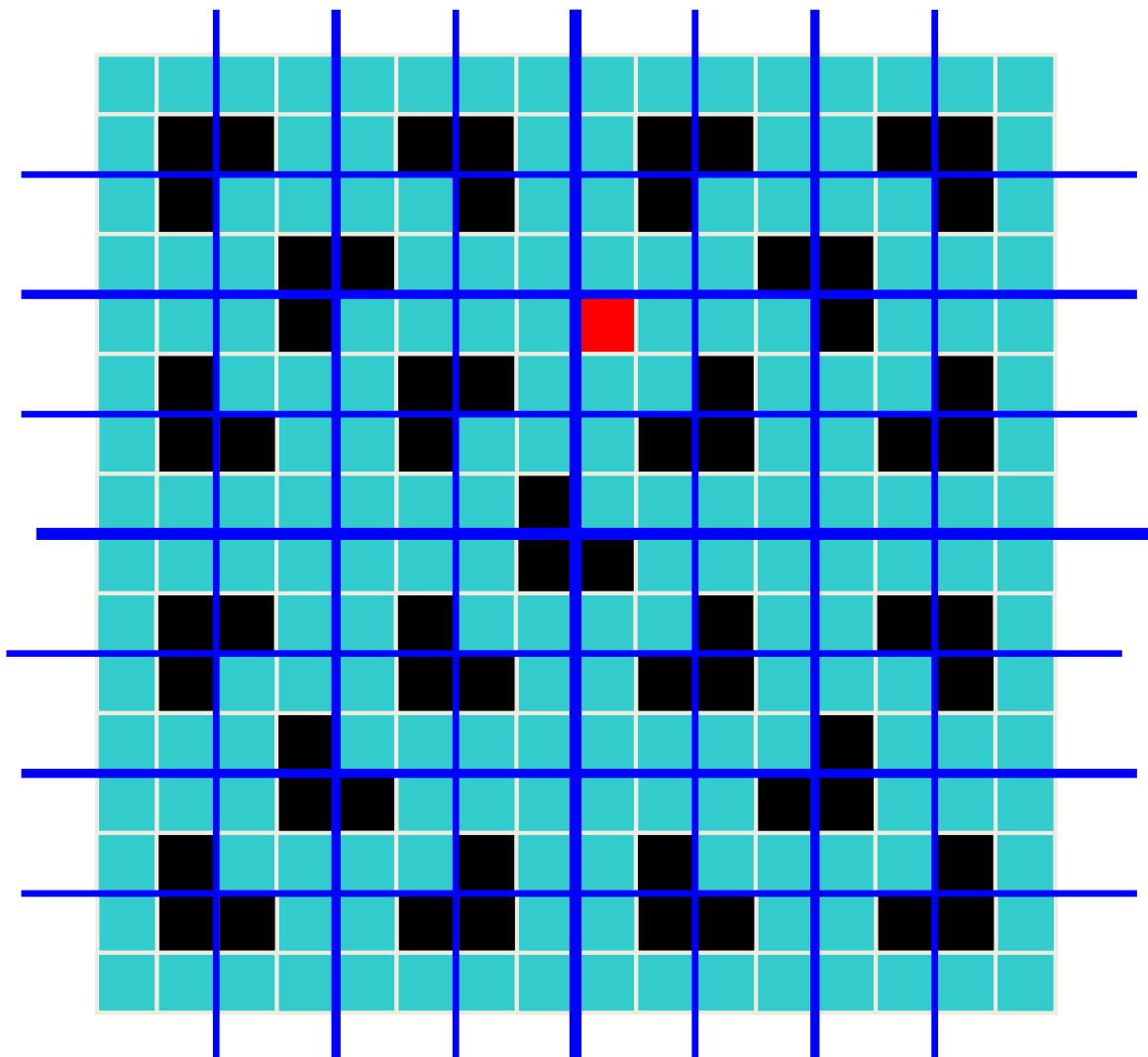
第一次分割



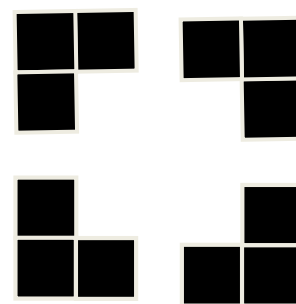
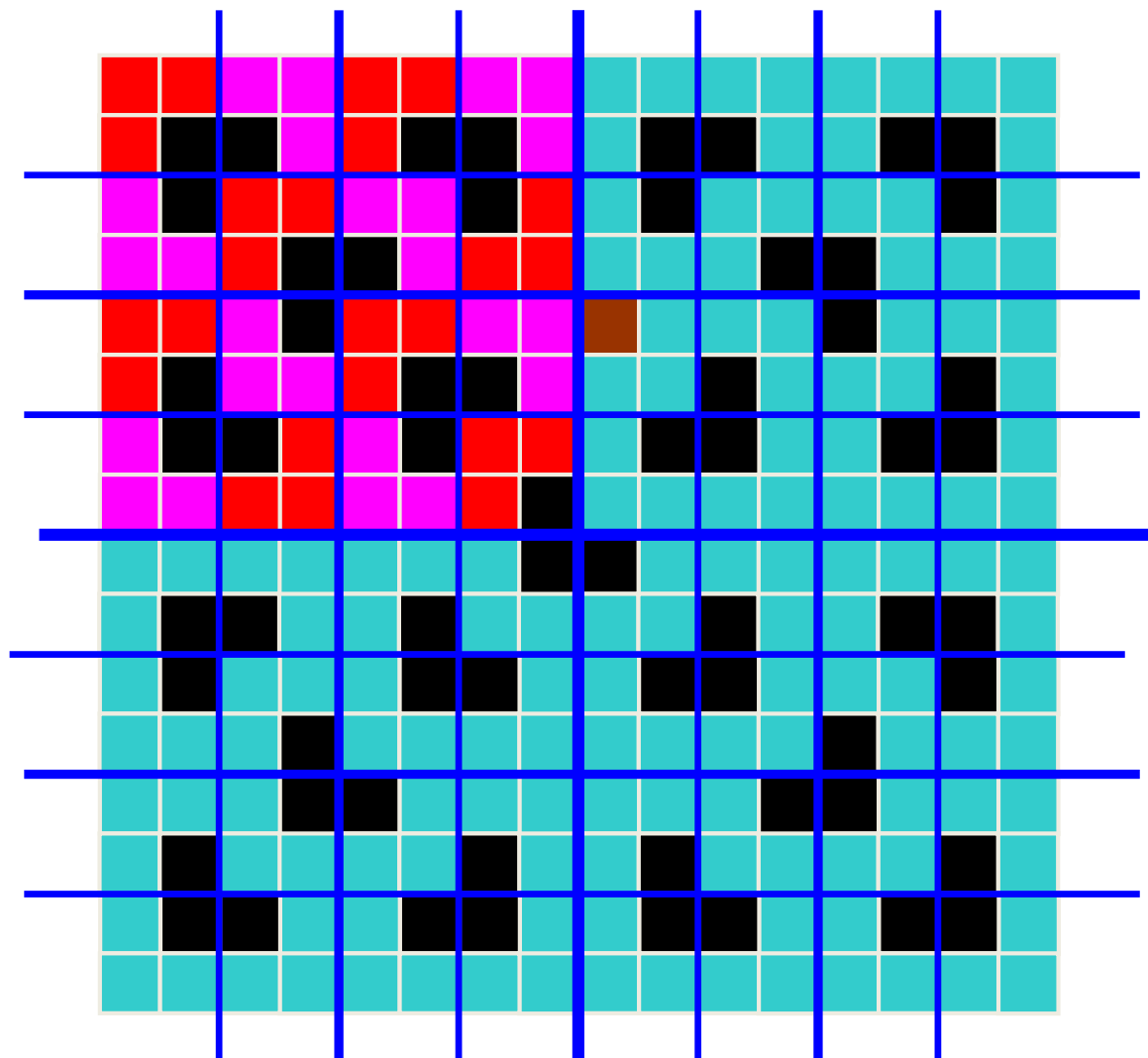
第二次分割



第三次分割



第四次分割为 1×1 棋盘



深色的方格块是人为填充的；剩下的方格块是自动填充的

棋盘覆盖算法

算法:

ChessBoard(tr, tc, dr, dc, k)

全局变量:

tile = 1// 被覆盖的方格记号

Board[2^k][2^k]=0//存储棋盘覆盖标记

输入:

tr, tc //棋盘左上角行号, 列号

dr,dc //特殊方格左上角行号, 列号

k //棋盘尺寸 $2^k \times 2^k$

执行:

ChessBoard(0,0, dr, dc, k)

棋盘覆盖算法

过程: ChessBoard(tr, tc, dr, dc, k)

IF $k = 0$

return

$t \leftarrow \text{tile}++$ //被覆盖的方格记号

$k \leftarrow k-1$ //分割棋盘

//求解左上角子棋盘

IF $dr < tr+2^k$ AND $dc < tc+2^k$ //特殊方格在此棋盘中

ChessBoard(tr, tc, dr, dc, k)

ELSE

Board[tr+2^k-1][tc+2^k-1] $\leftarrow t$ //标记覆盖右下角

ChessBoard(tr, tc, tr+2^k-1, tc+2^k-1, k)

//求解右上角子棋盘

IF $dr < tr+2^k$ AND $dc \geq tc+2^k$ //特殊方格在此棋盘中

ChessBoard(tr, tc+2^k, dr, dc, k)

ELSE

Board[tr+2^k-1][tc+2^k] $\leftarrow t$ //标记覆盖左下角

ChessBoard(tr, tc+2^k, tr+2^k-1, tc+2^k, k)

//求解左下角子棋盘

IF $dr \geq tr+2^k$ AND $dc < tc+2^k$ //特殊方格在此棋盘中

ChessBoard(tr+2^k, tc, dr, dc, k)

ELSE

Board[tr+2^k][tc+2^k-1] $\leftarrow t$ //标记覆盖右上角

ChessBoard(tr+2^k, tc, tr+2^k, tc+2^k-1, k)

//求解右下角子棋盘

IF $dr \geq tr+2^k$ AND $dc \geq tc+2^k$ //特殊方格在此棋盘中

ChessBoard(tr+2^k, tc+2^k, dr, dc, k)

ELSE

Board[tr+2^k][tc+2^k] $\leftarrow t$ //标记覆盖左上角

ChessBoard(tr+2^k, tc+2^k, tr+2^k, tc+2^k, k)

算法分析

建立递归方程：

$$T(k) = \begin{cases} \Theta(1) & k = 1 \\ 4T(k-1) + \Theta(1) & k > 1 \end{cases}$$

时间复杂度：

$$\begin{aligned} T(k) &= 4T(k-1) + \Theta(1) \\ &= 4(4T(k-2) + \Theta(1)) + \Theta(1) \end{aligned}$$

⋮

$$= 4^k T(0) + \Theta(1) \sum_{i=0}^{k-1} 4^i$$

$$= \Theta(4^k)$$

人为放置L型骨牌